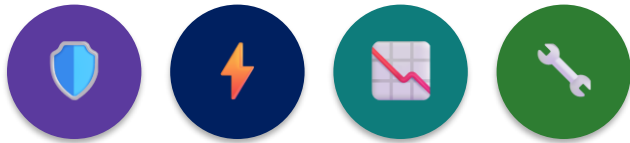


DAY 3

Resilience, Security, and Testing

Secured, resilient, tested Account and Transaction APIs with safe fallbacks. · Lab 3 — Secure and Resilient Services

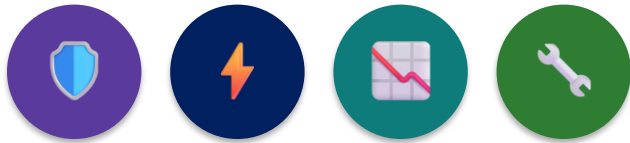




MODULE 6

Resilience and Performance

Keep services up when dependencies fail: timeouts, retries, circuit breakers, bulkheads, caching, and the SLOs a bank will actually hold you to.



DESIGNING FOR FAILURE

Services may be unavailable, slow, or overloaded -- resilience means defining what happens next, not hoping it never occurs.



ASSUME FAILURE IS NORMAL

- Services may be unavailable, responses may be slow, databases may reject connections
- Message brokers may back up, external APIs may throttle, infrastructure may restart



RESILIENCE BUILDING BLOCKS

- Timeouts, retries, circuit breakers, bulkheads, rate limits
- Health checks, metrics, logging, distributed tracing, alerting

KEY TAKEAWAY Services may be unavailable, slow, or overloaded -- resilience means defining what happens next, not hoping it never occurs.

Day 3 Objectives and Capstone Progress

DAY 3 OBJECTIVES

-  **1 Risk Service Development**
Build the Risk Service with rules engine and scoring logic.
-  **2 Advanced Rules & Scoring**
Implement configurable risk rules and scoring algorithms.
-  **3 Event-Driven Processing**
Consume transaction events and publish risk evaluation results.
-  **4 Notification Service**
Build Notification Service to send alerts based on risk results.
-  **5 End-to-End Saga Flow**
Complete the saga flow: Transaction → Risk Evaluation → Notification.
-  **6 Testing & Observability**
Add tests, verify integrations, and monitor services with Actuator.

CAPSTONE PROGRESS

Overall Progress

60%



What We Will Achieve by End of Day 3



Risk evaluation engine working



Rules configuration and scoring



Events consumed and published



Alerts generated for high-risk transactions



End of Day 3 Goal

Complete Risk Service and integrate with Transaction Service.
Have risk evaluation results published and ready for notifications.

FAILURE MODES IN DISTRIBUTED SYSTEMS

Immediate errors, slow responses, timeouts, and partial failures each demand a different, deliberately designed response.

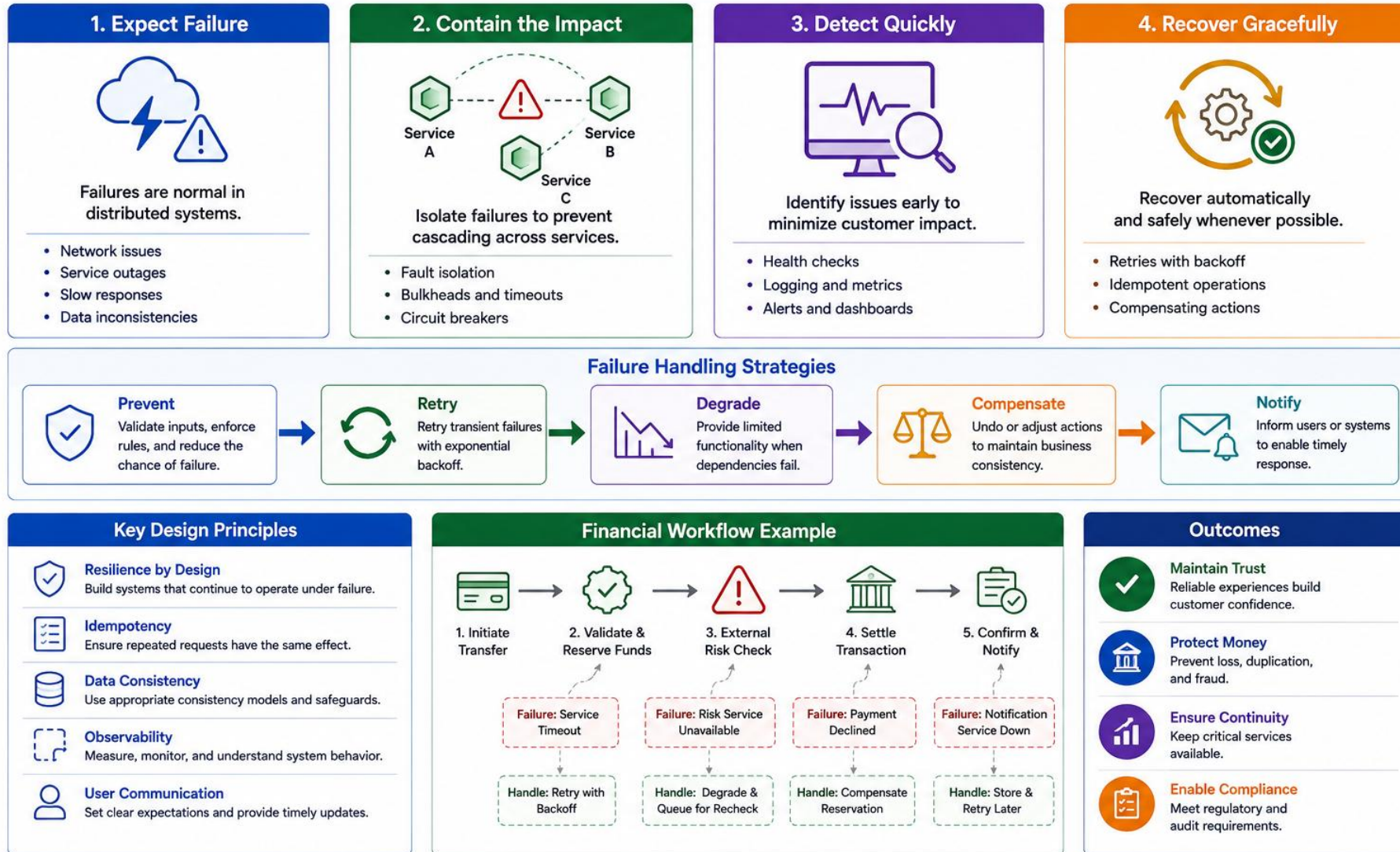


COMMON FAILURE TYPES

- Immediate error response, slow response, timeout, partial failure
- Resource exhaustion, message processing failure, data inconsistency, configuration error

KEY TAKEAWAY Immediate errors, slow responses, timeouts, and partial failures each demand a different, deliberately designed response.

Designing for Failure in Financial Workflows



NETWORK LATENCY AND UNRELIABLE COMMUNICATION ·

CASCADING FAILURE



WHY NETWORK CALLS ARE DIFFERENT

- Latency, jitter, and packet loss are inherent to any network hop
- A remote call can fail in ways a local call simply cannot: DNS failure, connection reset, partial response

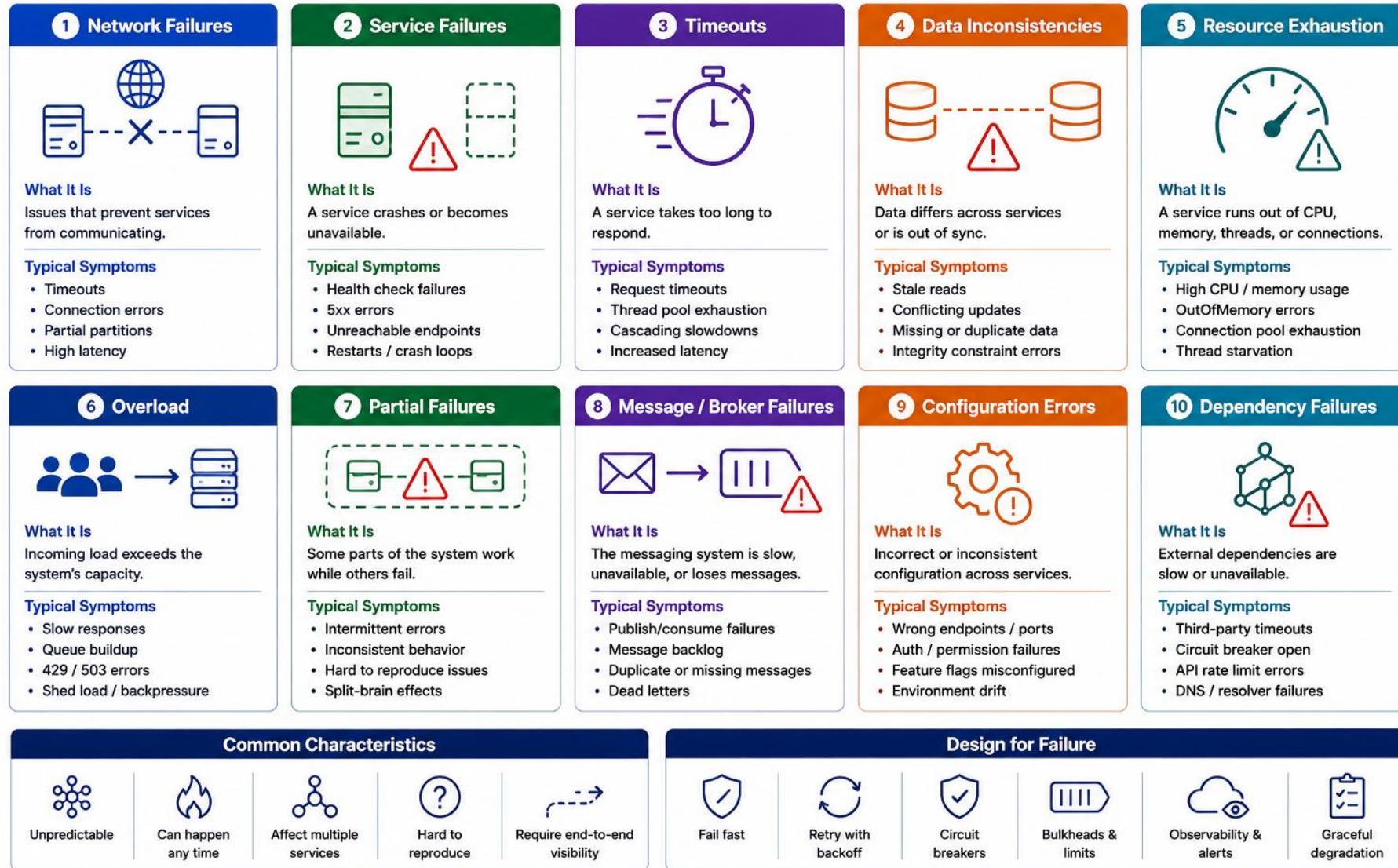


HOW CASCADING FAILURE SPREADS

- A slow or failing dependency ties up threads, connections, and memory in the caller
- The caller becomes unhealthy too, and the failure propagates to ITS callers
- Timeouts, circuit breakers, and bulkheads exist specifically to stop this chain

KEY TAKEAWAY In a monolith, calls happen in memory; in microservices, every call crosses a network that can delay, drop, or corrupt it. • A slow Fraud Detection Service can exhaust Transaction Service's threads, which then can't serve any request -- healthy or not.

Distributed-System Failure Modes



TIMEOUT PATTERN

Without a timeout, slow dependencies can silently consume threads, memory, and connection pools until the caller itself becomes unhealthy.



WHERE TIMEOUTS APPLY

- HTTP calls, database queries, message broker operations, external API calls
- File operations and cache operations also need timeouts



BANKING EXAMPLE

- Transaction Service -> Account Service: waits up to 2 seconds
- If no response, timeout handling takes over -- Transaction Service does not wait indefinitely

KEY TAKEAWAY Without a timeout, slow dependencies can silently consume threads, memory, and connection pools until the caller itself becomes unhealthy.

Timeout Strategies

1. Timeout Types



Connection Timeout

Maximum time to establish a connection.



Read Timeout

Maximum time to wait for a response after request is sent.



Overall Timeout

Total time allowed for the entire operation.

2. Recommended Approach



Set Timeouts Appropriately

Based on service behavior and SLA requirements.



Differentiate by Operation

Use different timeouts for read, write, and health checks.



Fail Fast

Use reasonable timeouts to free resources quickly.



Retry or Fallback

Combine timeouts with retry or fallback strategies.



Observe & Tune

Monitor latency and adjust timeouts over time.

3. Timeout Configuration Guidelines

Timeout Type	Typical Range	Guideline
 Connection Timeout	1 – 3 seconds	Keep low to quickly detect network or service issues.
 Read Timeout	2 – 10 seconds	Based on expected processing time and percentiles (P95/P99).
 Overall Timeout	= Connection + Read + Buffer	Include buffer for retries and downstream calls.

Best Practices

- ✓ Set timeouts shorter than user or upstream timeouts.
- ✓ Use different timeouts for internal vs external calls.
- ✓ Avoid very long timeouts that tie up resources.
- ✓ Document timeout values and review regularly.

4. Timeout + Retry + Backoff



Request with Timeout



Timeout Occurs



Retry with Backoff



Success (or Fallback)



Use exponential backoff with jitter to reduce load and avoid retry storms.

5. Example Configuration (Spring Boot)

```
# Example for RestTemplate / WebClient
connect-timeout: 2000    # 2 seconds
read-timeout: 5000      # 5 seconds

# Example for WebClient
response-timeout: 5s     # 5 seconds

# Resilience4j Timeout
resilience4j.timeout.instances.myService.
  timeout-duration: 3s
  cancel-running-future: true
```

6. Outcome



Prevents cascading failures



Improves system responsiveness



Frees up resources quickly



Better user experience



Key Takeaway

Use appropriate timeouts, combine with retries and fallbacks, and continuously observe and tune to build resilient and responsive distributed systems.

CHOOSING EFFECTIVE TIMEOUT VALUES · RETRY PATTERN · EXPONENTIAL BACKOFF AND JITTER



TIMEOUT BUDGET EXAMPLE

- Total budget: 3000 ms
- Customer lookup: 500 ms, Account validation: 700 ms, Fraud check: 800 ms
- Remaining budget covers response processing



SELECTION FACTORS

- Business requirements, expected latency, user experience, downstream behavior, SLA targets
- Short timeouts fail fast but risk false rejects; long timeouts tie up resources



RETRYABLE FAILURES

- Temporary network error, HTTP 503, HTTP 429 with retry-after guidance
- Short database connection interruption, temporary broker disconnect



NON-RETRYABLE FAILURES

- Validation errors, authentication failure, authorization failure
- Resource not found, business rule rejection



BACKOFF PROGRESSION EXAMPLE

- Attempt 1 fails -> wait 200 ms
- Attempt 2 fails -> wait 500 ms
- Attempt 3 fails -> stop and report failure



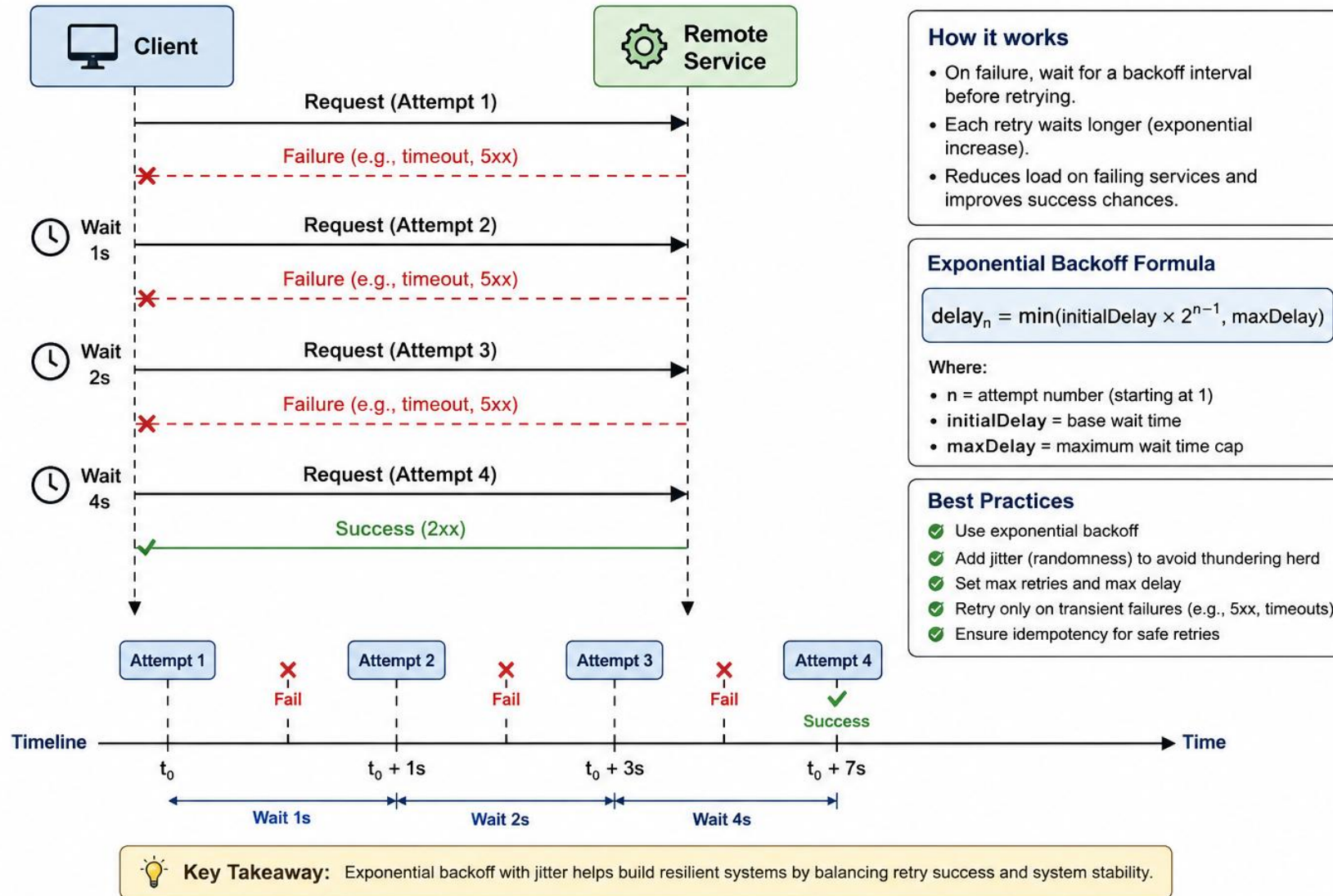
WHY JITTER MATTERS

- Adds randomness to retry delays so many instances don't retry in lockstep
- Prevents a coordinated retry spike from overwhelming a recovering dependency

KEY TAKEAWAY If the customer-facing API must respond in 3 seconds, downstream timeouts must be carved out of that same 3-second budget.

- Retrying a temporary email provider outage is safe; retrying a money transfer without idempotency can create duplicate transactions.
- Without jitter, many service instances retrying on the exact same schedule can create a synchronized retry storm that re-overloads the dependency.

Retry Mechanisms and Exponential Backoff (awareness)



WHEN RETRIES MAKE FAILURES WORSE · CIRCUIT BREAKER PATTERN



HOW RETRIES BACKFIRE

- Retrying every exception, including non-transient ones
- No maximum retry limit, or retrying non-idempotent operations
- Combining long timeouts with multiple retries multiplies total wait time



WHY CIRCUIT BREAKERS HELP

- Prevent cascading failures, thread exhaustion, and excessive retries
- Stop slow request buildup and avoid overloading an already-failing service

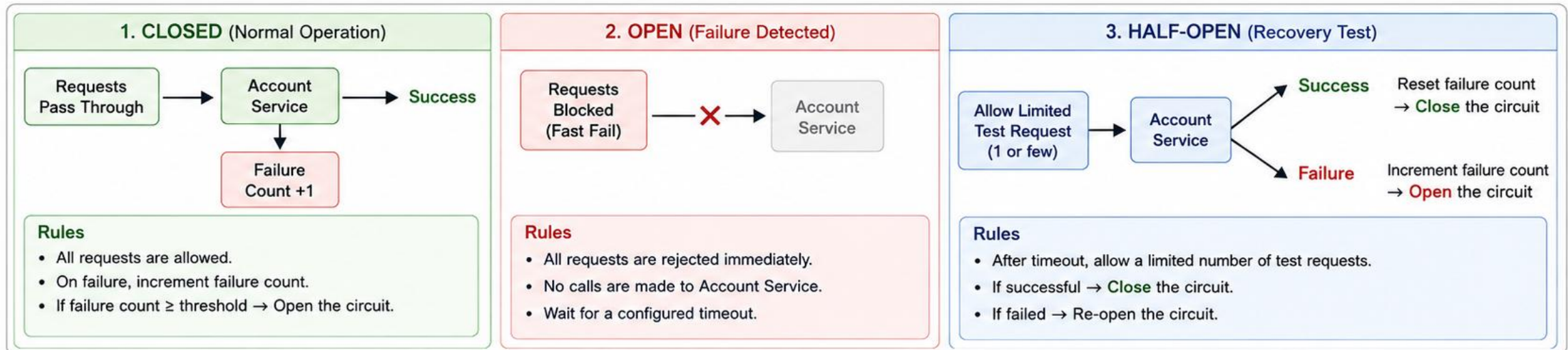
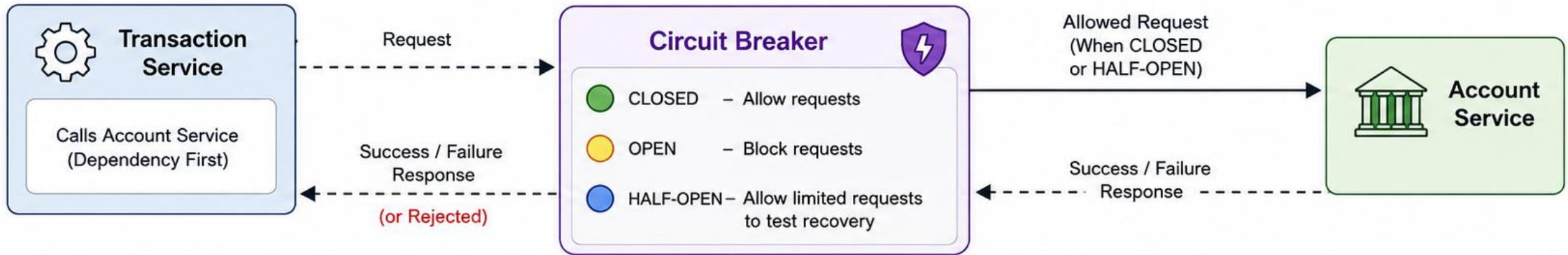


BANKING EXAMPLE

- If Fraud Detection Service fails repeatedly, the circuit opens
- Transactions may be rejected, marked pending review, or handled under a fallback policy

KEY TAKEAWAY Retrying every exception, with no limit, from many instances at once, amplifies load on a dependency that's already struggling to keep up. • When failures exceed a threshold, the circuit opens and blocks calls immediately -- protecting both the caller's resources and the struggling dependency.

Circuit Breaker Pattern (Account Service dependency first)



CIRCUIT BREAKER STATES AND TRANSITIONS · BULKHEAD PATTERN



THE THREE STATES

- Closed: all calls allowed, success/failure rates monitored
- Open: calls blocked immediately, fail fast or use a fallback
- Half-open: a small number of test calls allowed after a wait duration



RESILIENCE4J CONFIGURATION CONCEPT

- failure-rate-threshold: 50, sliding-window-size: 20, wait-duration-in-open-state: 30s

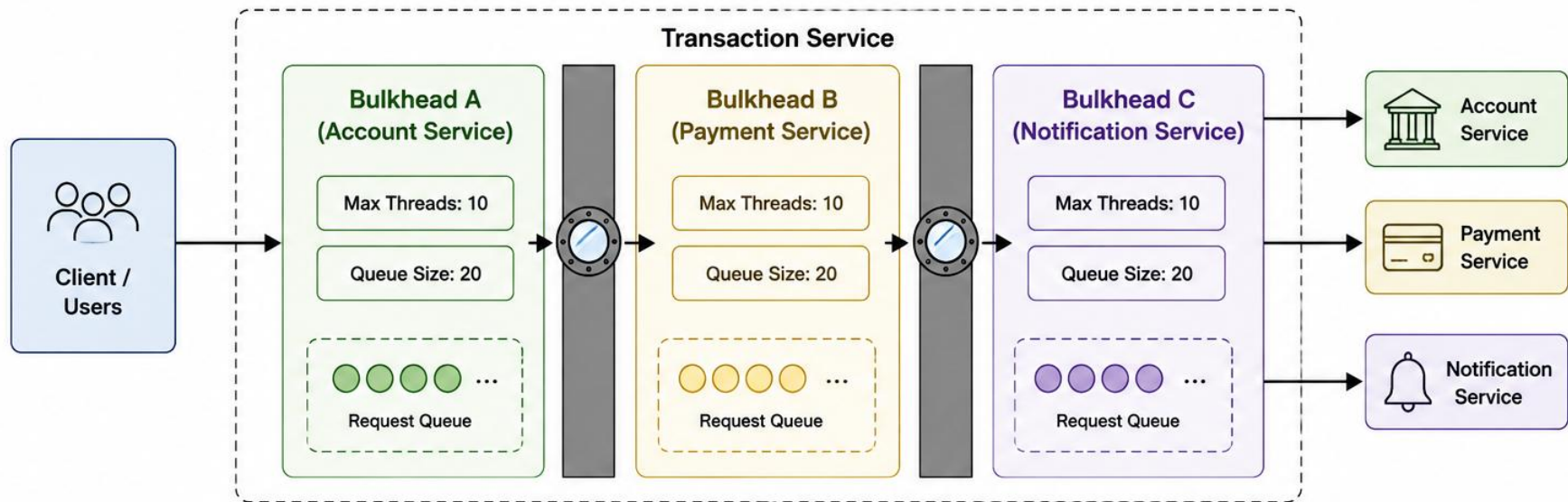


WHY BULKHEADS MATTER

- Without isolation, one slow dependency can consume all available threads
- The term comes from ship compartments; in software it means separate thread pools, connection pools, or concurrency limits

KEY TAKEAWAY Half-open allows a small number of test calls through -- success closes the circuit, failure reopens it immediately. • If reporting export hangs, Account Service should still serve balance lookups -- bulkheads are what make that guarantee possible.

Bulkhead Pattern (conceptual)



How It Works

1

Each downstream dependency is isolated behind its own bulkhead with separate resource limits.

2

If one dependency is slow or fails, its bulkhead exhausts resources independently.

3

Other bulkheads still have available resources and continue to serve requests.

4

Failures are contained to the failing bulkhead; system remains resilient.

Key Purpose



Prevent a failure or high load in one external dependency from exhausting resources and impacting others.

Common Resource Limits



• Max Threads / Connections



• Queue Size



• Timeout per Call



• Rate Limit (optional)



Result: Improved isolation, better fault tolerance, and predictable performance.

SEMAPHORE VS. THREAD-POOL BULKHEADS · RATE LIMITING AND THROTTLING · FALLBACK STRATEGIES



TWO BULKHEAD IMPLEMENTATIONS

- Semaphore bulkhead: limits concurrent access without a separate thread pool, lightweight
- Thread pool bulkhead: uses a separate thread pool, can queue calls before rejection



WHERE IT'S APPLIED

- API gateway, load balancer, service mesh, application code, or the external API client
- Exceeding the limit returns 429 Too Many Requests



WHEN TO USE EACH

- Semaphore: reactive or non-blocking applications
- Thread pool: blocking operations that need full isolation of thread resources



FALLBACK OPTIONS

- Return cached reference data, mark a transaction as pending, return a controlled error
- Queue work for later processing, or disable a non-critical feature



COMMON ALGORITHMS

- Fixed window, sliding window, token bucket, leaky bucket
- Token bucket: tokens refill over time; each request consumes one, allowing controlled bursts



THE GOLDEN RULE

- Fallbacks must be business-approved and must never invent or fabricate critical data

KEY TAKEAWAY A semaphore is lightweight and non-blocking by default; a thread pool bulkhead adds true isolation at the cost of managing its own threads. • Fixed window, sliding window, token bucket, and leaky bucket algorithms each shape traffic differently, but all exist to protect capacity. • Returning a fabricated account balance is never acceptable; hiding a marketing offer when the recommendation service is down usually is.

GRACEFUL DEGRADATION · LOAD SHEDDING · RESILIENCE4J OVERVIEW



WHAT GRACEFUL DEGRADATION MEANS

- The system provides reduced functionality instead of failing completely
- For critical banking operations, degradation must still follow business and compliance rules



HOW LOAD SHEDDING WORKS

- The service monitors its own saturation (CPU, threads, queue depth)
- When capacity is exceeded, lower-priority requests are rejected early, before they consume resources
- Protects the service's ability to serve its most important traffic



WHY RESILIENCE4J

- Lightweight and fast, fault tolerant by design
- Rich metrics and events, production ready
- Modules for Retry, CircuitBreaker, Bulkhead, RateLimiter, and TimeLimiter

KEY TAKEAWAY If Recommendation Service is unavailable, the dashboard still shows balances and transactions -- it simply hides personalized offers.

- When demand exceeds capacity, shedding low-priority traffic protects the requests that matter most from a total system collapse.
- Timeout, retry, circuit breaker, bulkhead, and rate limiter modules share one consistent configuration style across a Spring Boot service.

IMPLEMENTING TIMEOUTS WITH RESILIENCE4J · IMPLEMENTING RETRIES WITH RESILIENCE4J · IMPLEMENTING CIRCUIT BREAKERS



CONFIGURATION CONCEPT

- `resilience4j.timelimiter.instances.accountService.timeout-duration: 2s`
- `@TimeLimiter(name = "accountService")` on the client method
- Pairs naturally with a fallback method for the timeout case



SPRING BOOT EXAMPLE

- `@CircuitBreaker(name = "externalService", fallbackMethod = "fallback")`
- `sliding-window-size`, `minimum-number-of-calls`, `failure-rate-threshold`, `wait-duration-in-open-state`



CONFIGURATION CONCEPT

- `resilience4j.retry.instances.accountService.max-attempts: 3`
- `resilience4j.retry.instances.accountService.wait-duration: 500ms`
- Combine with exponential backoff and jitter, and target only transient failures



METRICS AND EVENTS

- Micrometer auto-registers `resilience4j.circuitbreaker.calls`, `.state`, `.successful.calls`, and more
- Key events: `OnSuccess`, `OnError`, `OnSlowCall`, `OnCallNotPermitted`, `OnStateTransition`

KEY TAKEAWAY `@TimeLimiter` combined with a fallback method turns a hung downstream call into a controlled, predictable failure instead of an indefinite wait. • `max-attempts` and `wait-duration` in `application.yml` are all it takes to add safe, bounded retry behavior to a Spring Boot client call. • A sliding window of recent calls, a failure-rate threshold, and a `wait-duration-in-open-state` fully define a Resilience4j circuit breaker.

IMPLEMENTING BULKHEADS · IMPLEMENTING RATE LIMITERS · COMBINING RESILIENCE PATTERNS SAFELY



CONFIGURATION EXAMPLE

- `resilience4j.bulkhead.instances.externalService.max-concurrent-calls: 10`
- `type: SEMAPHORE` or `THREADPOOL`, with queue-capacity for thread-pool bulkheads
- Use `BulkheadFullException` to provide a graceful fallback when capacity is exhausted



RECOMMENDED EXECUTION ORDER

1. Timeout -- fail fast
2. Circuit Breaker -- stop failing fast
3. Retry -- recover from transient failures
4. Bulkhead -- isolate and contain
5. Rate Limiter -- control traffic
6. Fallback -- provide graceful behavior



CONFIGURATION EXAMPLE

- `resilience4j.ratelimiter.instances.externalService.limit-for-period: 10`
- `limit-refresh-period: 1s`, `timeout-duration: 500ms`, `timeout-behavior: WAIT` or `FAIL_FAST`



COMMON ANTI-PATTERNS

- Retrying non-idempotent operations, retries without backoff or jitter
- Unlimited bulkhead thread pools, overusing too many patterns at once

KEY TAKEAWAY `max-concurrent-calls` and `max-wait-duration` cap how many requests can be in flight to one dependency at any moment. • `limit-for-period` and `limit-refresh-period` define a token bucket; excess calls wait, or fail fast with `RequestNotPermittedException`. • Applying patterns in this recommended sequence protects the provider first, protects your own threads second, and degrades gracefully last.

RESILIENCE CONFIGURATION AND TUNING · TESTING FAILURE AND RECOVERY · CHAOS ENGINEERING FUNDAMENTALS



TUNING WITH DATA

- Collect: metrics, logs, and traces (p95/p99 latency, errors, timeouts)
- Analyze latency distributions and failure types; adjust one parameter at a time
- Validate improvements without introducing new issues, then document the decision



THE CHAOS ENGINEERING LIFECYCLE

1. Define hypotheses about expected system behavior
2. Inject controlled, safe failures (latency, pod kill, etc.)
3. Observe system response and identify weaknesses
4. Learn and improve; repeat continuously



TYPES OF FAILURES TO TEST

- Network failures, dependency failures, resource exhaustion
- Messaging failures, infrastructure failures (pod/node crashes)



COMMON EXPERIMENTS AND TOOLS

- Service kill, network latency, network partition, resource depletion, dependency failure
- Tools: Chaos Mesh, Litmus, Gremlin, Pumba, Toxiproxy



WHAT TO VALIDATE

- Do timeouts, retries, circuit breakers, bulkheads, and rate limiters behave as configured?
- Does the system recover automatically and quickly once the failure is removed?

KEY TAKEAWAY Measure, analyze, adjust one parameter at a time, validate, and document -- resilience tuning is a continuous feedback loop, not a one-time setup. • Plan, inject failure, observe, validate, recover -- a five-step approach turns 'we think it's resilient' into 'we know it's resilient.' • Define hypotheses, inject controlled failures, observe behavior, and learn -- turning unknown risks into known, fixable weaknesses.

RESILIENCE ANTI-PATTERNS · MICROSERVICES PERFORMANCE FUNDAMENTALS · LATENCY, THROUGHPUT, AND RESOURCE UTILIZATION

COMMON ANTI-PATTERNS

- Infinite retries with no backoff; circuit breakers tuned so aggressively they stay open
- Universal fallbacks that return stale or incorrect data and hide real failures
- Deep synchronous call chains, no timeouts anywhere, oversized bulkheads with no isolation

HOW TO AVOID THEM

- Understand your dependencies, tune based on real data, and test under actual failure
- Instrument everything -- failing silently is not the same as being resilient

COMMON PERFORMANCE BOTTLENECKS

- Network calls (latency, chattiness), database queries (N+1, missing indexes)
- Thread pool sizing, caching (low hit rate, stampede), serialization overhead

BEST PRACTICES

- Design for loose coupling, use asynchronous messaging where possible
- Cache close to the data, right-size pools, paginate, set timeouts, profile hot paths

THE THREE DIMENSIONS

- Latency: how fast a request is served (p50, p95, p99)
- Throughput: how much work is done per unit time (requests per second)
- Resource utilization: how efficiently CPU, memory, disk, and network are used

TRADE-OFFS TO UNDERSTAND

- Lower latency may reduce throughput; higher throughput may consume more resources
- Keep headroom -- never run at 100% utilization

KEY TAKEAWAY Infinite retries, always-open circuit breakers, and universal fallbacks that lie are common ways good intentions produce bad outcomes. • Performance is a feature -- optimizing end-to-end user experience matters more than tuning any single service in isolation. • Lowering latency often needs more resources; pushing throughput higher needs efficient resource usage -- optimizing one affects the others.

PERFORMANCE BUDGETS · CONNECTION POOLING AND THREAD MANAGEMENT · JVM PERFORMANCE CONSIDERATIONS



PERFORMANCE BUDGET FORMULA

- Performance Budget = Target (user expectation) - Safety Margin - Variability Buffer
- Types: latency budget, throughput budget, availability budget, error budget, resource budget



CONNECTION POOLING

- Limits the number of open connections to a safe, efficient size
- Key settings: `maximumPoolSize`, `minimumIdle`, `idleTimeout`, `connectionTimeout`, `maxLifetime`



THREAD MANAGEMENT

- Too few threads means low throughput; too many means context switching and CPU pressure
- Use bounded thread pools and tune based on the actual workload



KEY AREAS THAT IMPACT PERFORMANCE

- Memory management and garbage collection tuning prevent `OutOfMemoryError` and reduce pauses
- JIT compilation, thread management, and blocking I/O all shape throughput and latency



GC OPTIONS

- G1 (default): balanced throughput and pause times, good for large heaps
- ZGC / Shenandoah: very low pause times for latency-sensitive services

KEY TAKEAWAY If the end-to-end target is 500 ms, each service in the chain gets its own slice -- and the sum must never exceed the total budget. • Creating a new database connection is expensive -- a connection pool reuses existing ones and prevents connection storms. • G1 is the sensible default for most microservices; ZGC and Shenandoah trade some throughput for very low pause times when latency is critical.

HORIZONTAL VS. VERTICAL SCALING · BACKPRESSURE AND FLOW CONTROL · LOAD TESTING MICROSERVICES



HORIZONTAL SCALING (SCALE OUT)

- Add more instances behind a load balancer; distributes load across many nodes
- High availability and fault tolerance, but requires stateless applications



VERTICAL SCALING (SCALE UP)

- Add more CPU, RAM, and storage to a single node; simple, no code changes
- Hits a hardware ceiling and can require downtime to apply



HOW BACKPRESSURE WORKS

- Signals flow upstream to slow down the producer when the consumer or queue is overwhelmed
- Prevents buffer overflow and out-of-memory errors, keeping the system stable under load



FLOW CONTROL TECHNIQUES

- Rate limiting, bounded queues, credit-based flow control, reactive streams



KEY METRICS TO MONITOR

- Response time (avg, p95, p99), throughput, and error rate
- CPU/memory utilization, queue depth/lag, active connections and threads

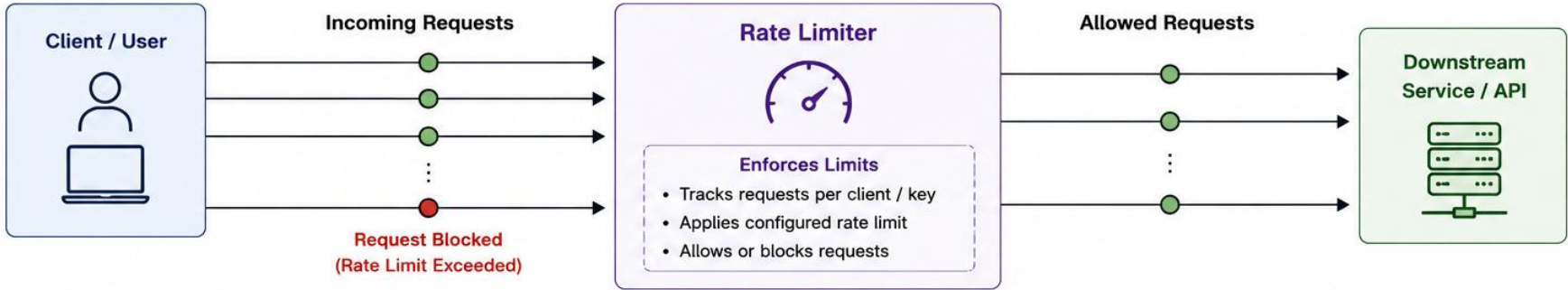


COMMON TOOLS

- k6, JMeter, Gatling, Locust, Artillery, NBomber

KEY TAKEAWAY Horizontal scaling adds more instances behind a load balancer; vertical scaling adds more CPU, RAM, and storage to a single node. • Without backpressure, unbounded queues grow until they exhaust memory; with it, the whole pipeline degrades gracefully instead of crashing. • Define objectives, plan, design, execute, analyze, then optimize and retest -- a six-step process turns load testing into a repeatable discipline.

Rate Limiting Overview (conceptual)



Common Rate Limiting Algorithms

1. Fixed Window Counter

Counts requests in fixed time windows. Simple but can allow bursts at window boundaries.

2. Sliding Window Log

Stores timestamps of requests in the current window. More accurate but uses more memory.

3. Sliding Window Counter

Prev. Window	Current Window
60 / 100	40 / 100

Weighted count of current and previous windows. Balances accuracy and efficiency.

4. Token Bucket

Tokens added at a steady rate. Requests consume tokens. Allows bursts up to bucket capacity.

Key Benefits

- ✓ Prevents abuse and protects system resources
- ✓ Ensures fair usage among clients
- ✓ Enhances stability and performance
- ✓ Supports multi-tenant and API monetization

Common Use Cases

- API protection
- Per-user / per-tenant limits
- Prevent brute force / abuse
- Control traffic to downstream services

Typical Configuration

Scope / Key	e.g., IP address, User ID, API Key
Limit	e.g., 100 requests
Time Window	e.g., per minute
Action on Limit Exceeded	e.g., Reject (429), Throttle, Queue
Headers	e.g., Retry-After, X-RateLimit-*

Rate Limit Response Example

HTTP Status: 429 Too Many Requests

Headers:
Retry-After: 60
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1715698800

Body (example):
{
 "error": "rate_limit_exceeded",
 "message": "Too many requests. Please try again later."
}

STRESS, SPIKE, SOAK, AND SCALABILITY TESTING · SERVICE-LEVEL INDICATORS · SERVICE-LEVEL OBJECTIVES

THE FOUR TEST TYPES

- Stress: push beyond capacity until failure occurs, to find the breaking point
- Spike: sudden, extreme load increases, to validate graceful handling
- Soak (endurance): sustained load for 24-72+ hours, to catch memory/resource leaks
- Scalability: increase resources and measure throughput improvement

KEY CHARACTERISTICS OF GOOD SLOs

- User-focused, measurable, time-bound, challenging but achievable
- Limited in number -- a few critical objectives per service, not dozens

COMMON SLIS FOR MICROSERVICES

SLI	MEASURED AS
Latency	p50, p95, p99 (ms)
Availability	% of successful requests
Error rate	% of requests resulting in errors
Saturation	CPU %, memory %, queue size
Traffic/Throughput	Requests per second (RPS)

BANKING EXAMPLE

- Payment Service latency (p95): under 300 ms over a 7-day window
- Order Service availability: 99.9% over a 30-day window

KEY TAKEAWAY Stress finds the breaking point, spike checks sudden surges, soak reveals slow leaks, and scalability testing measures how well growth is absorbed. • An SLI is a quantitative measurement from the user's point of view -- the raw data every SLO and error budget is built on top of. • 99.9% availability over 30 days means: we expect the service to be available 99.9% of the time each month -- a clear, actionable commitment.

Service-Level Agreements

Defining a Clear Contract for Service Reliability and Performance

A **Service-Level Agreement (SLA)** is a contract between a service provider and its consumers that defines the expected level of service.



SLAs create expectations, drive accountability, and enable reliable microservice operations.

WHY SLAs MATTER



Align expectations between providers and consumers



Drive reliability, quality, and consistency



Support capacity planning and performance goals

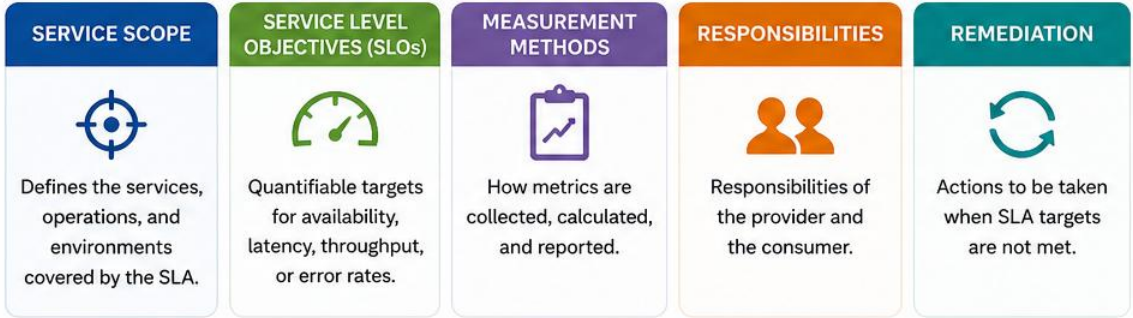


Define responsibilities and remediation commitments



Enable data-driven decisions and continuous improvement

KEY COMPONENTS OF AN SLA



EXAMPLE SLA		
	Service	Order Service
	Availability	99.9% monthly uptime
	Response Time (P95)	≤ 300 ms
	Error Rate	≤ 0.1% of requests
	Incident Response	Within 15 minutes
	Resolution Time	Within 4 hours
	Reporting	Monthly SLA reports
	Remedies	Service credits per policy



SLA defines the expected service levels and the commitments between parties.

SLA BEST PRACTICES



Set realistic and measurable goals.



Base SLAs on historical data and capacity.



Keep SLAs clear, simple, and understandable.



Monitor continuously and review regularly.



Communicate and collaborate across teams.



Use SLAs to drive improvement, not just compliance.

ERROR BUDGETS · CAPACITY PLANNING · PERFORMANCE TESTING IN CI-CD



HOW IT WORKS

1. Define an SLO, e.g. 99.9% availability over a month
2. Calculate the error budget: 0.1% of the time is allowed for errors
3. Monitor burn rate in real time
4. Use the budget to guide release velocity and reliability investment



CAPACITY HEADROOM GUIDELINES

- Low risk (stable demand): 10-20% headroom
- Mission critical: 50-100%+ headroom



CONSEQUENCES OF BUDGET STATUS

- Healthy: safe to release new features and take more risk
- Exhausted: freeze new features, focus on reliability and remediation



KEY CAPACITY DRIVERS

- User demand, business growth, seasonality, data growth
- Performance goals and resilience needs (HA, failover, disaster recovery)



PERFORMANCE TESTING IN THE CI/CD PIPELINE

- Code commit -> build -> static analysis -> integration tests -> performance tests -> quality gate
- Quality gate evaluates metrics against thresholds before promoting to staging or production



BEST PRACTICES

- Automate performance tests in every pipeline run; fail the build when thresholds are breached
- Track performance trends over time, not just pass/fail on a single run

KEY TAKEAWAY A 99.9% availability SLO over 30 days allows 43.2 minutes of downtime -- spend it wisely, and use burn rate to guide release risk. • Understand demand, collect metrics, forecast, decide, and implement -- a five-step process keeps infrastructure sized to real, not guessed, need. • A quality gate with pass/fail thresholds on response time, throughput, and error rate turns performance into a release blocker, not an afterthought.

Safe Fallback Rules - never invent a successful financial outcome

 Core Principle When a dependency is unavailable or uncertain, do not assume success. Return a safe failure or pending state.		Mindset → Don't guess. → Don't fabricate. → Don't create money, move money, or change balances.		 Golden Rule Fail safe. Be conservative. Protect financial integrity.
Scenario (Dependency Fails)	Never Do This	Do This Instead (Safe Fallback)	Client Response	
 Account Service unavailable (balance or account validation)	 Assume account exists and has sufficient funds. Do not proceed.	 Return 503/Service Unavailable or 424/Failed Dependency. Do not create or confirm transaction.	 Show failure to validate. Ask user to retry. No money moved.	
 Payment Gateway unavailable (charge authorization)	 Assume payment authorized. Do not capture/settle.	 Do not capture. Return 503/Service Unavailable. Keep transaction in PENDING state for retry or user action.	 Show payment pending or failed to process. No charge applied.	
 Fraud/Risk Service unavailable (risk evaluation)	 Assume low risk. Do not approve based on guess.	 Return 503/Service Unavailable or flag as PENDING_REVIEW. Do not auto-approve.	 Show pending review or try again later. No approval granted.	
 Ledger Service unavailable (posting transaction)	 Assume transaction posted. Do not record success.	 Do not mark as posted. Store in PENDING state. Retry posting; ensure idempotency.	 Show transaction pending. No success confirmation.	
 Notification Service unavailable (send receipt/alert)	 Assume notification delivered. Do not confirm sending.	 Accept the request. Store for retry (outbox). Do not mark as sent.	 Show notification pending. It will be sent later. No success confirmation.	
Safe Fallback Checklist  ✓ When in doubt, stop and fail safe. ✓ Return failure/pending, never success. ✓ Do not change balances or create commitments. ✓ Be explicit, transparent, and retryable.		 Remember A wrong success is worse than a temporary failure. Protect trust. Protect money.		

MODULE 7

Microservices Security

Secure service-to-service and user paths: OAuth2/OIDC, JWT, mTLS, secrets on OpenShift, and the controls that satisfy banking audit.



SECURITY THREATS IN DISTRIBUTED SYSTEMS

Unauthorized access, insecure service communication, injection, and misconfiguration top the list of threats a distributed architecture introduces.



COMMON THREATS

- Unauthorized access, insecure service-to-service communication, vulnerable components
- Data exposure, injection attacks, denial of service
- Supply chain attacks and misconfiguration

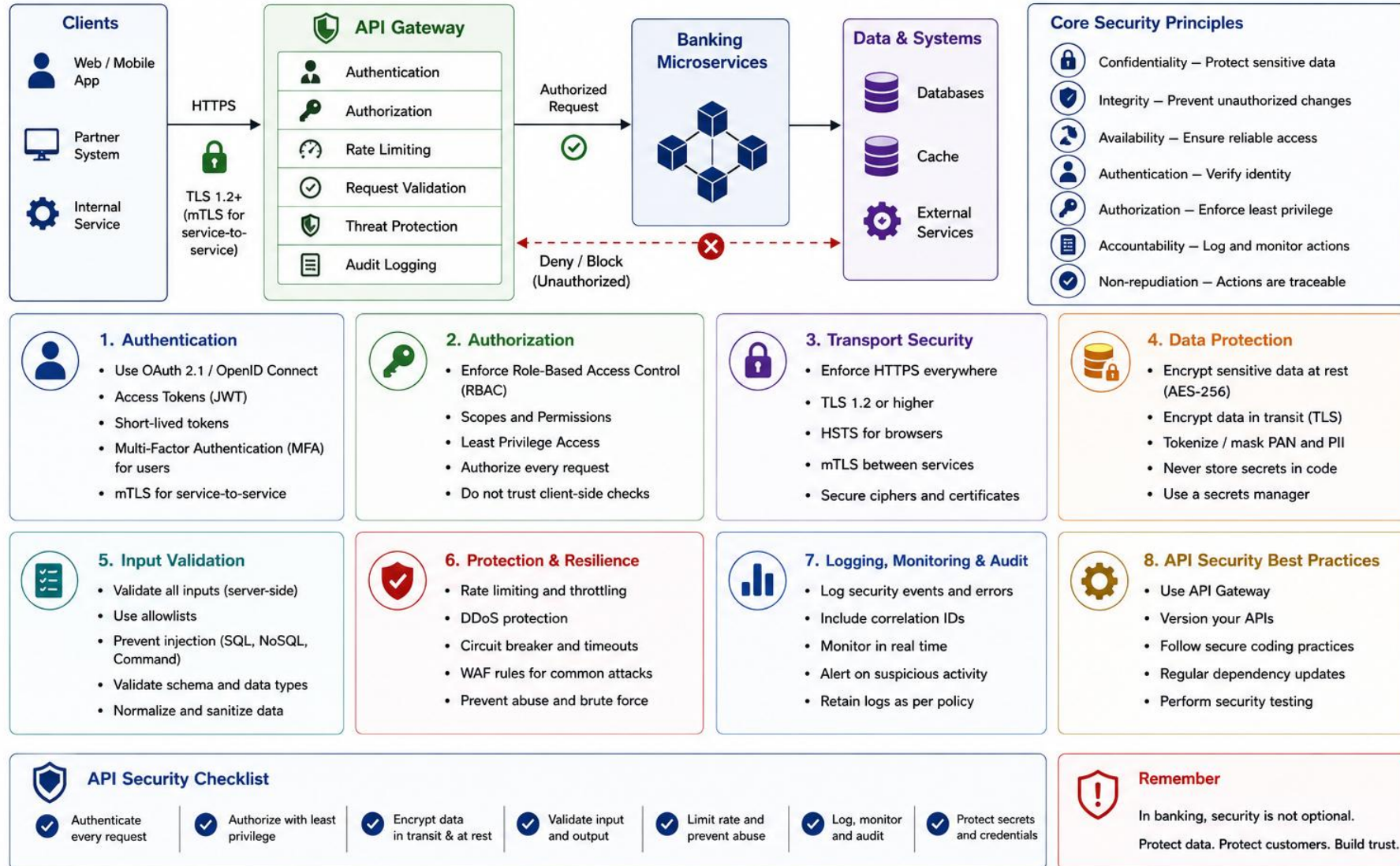


BEST PRACTICES TO REDUCE RISK

- Enforce strong authentication and authorization; encrypt data in transit and at rest
- Implement network policies, manage secrets securely, monitor and alert continuously

KEY TAKEAWAY Unauthorized access, insecure service communication, injection, and misconfiguration top the list of threats a distributed architecture introduces.

Security Fundamentals for Banking APIs



DEFENSE IN DEPTH FOR MICROSERVICES

Multiple, overlapping security controls across people, process, and technology



PRINCIPLE

Assume breaches will happen.
Layer multiple controls so that if one fails, others continue to protect your systems and data.

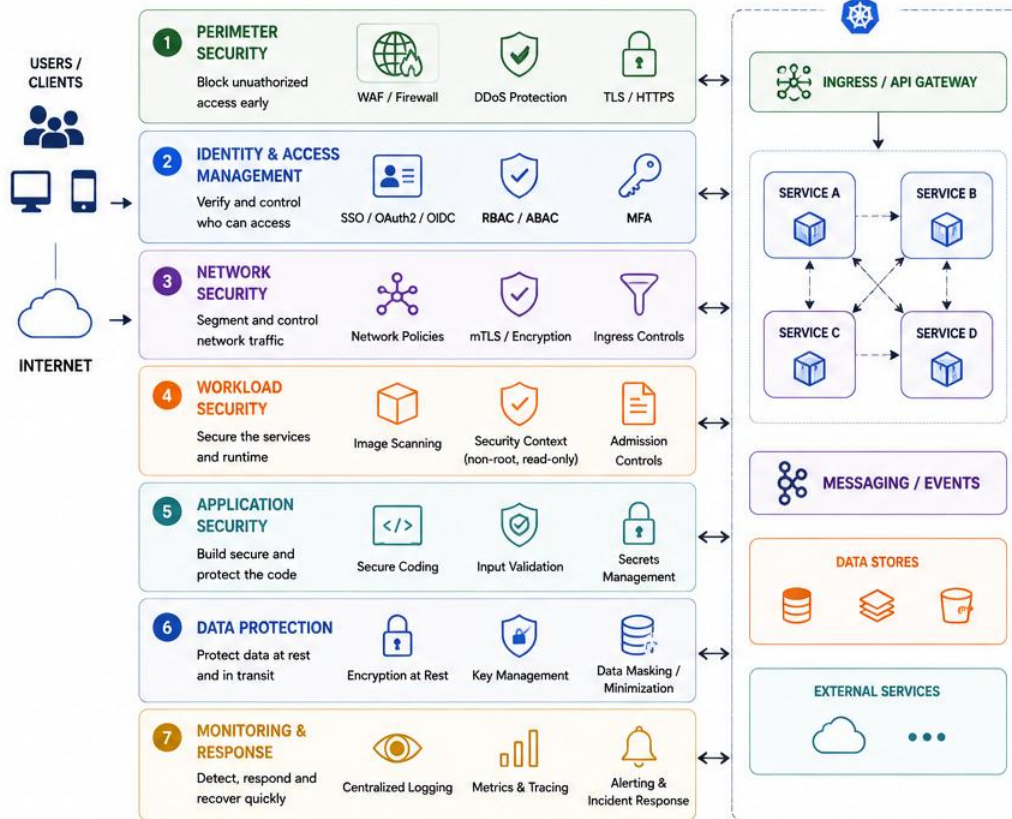
- ✓ No single point of failure
- ✓ Reduce blast radius
- ✓ Protect data and services
- ✓ Detect and respond quickly
- ✓ Continuous improvement



KEY TAKEAWAYS

- Apply controls at every layer of the microservices stack.
- Combine preventive, detective, and responsive controls.
- Automate and enforce security policies consistently.
- Monitor, audit, and improve continuously.

LAYERED SECURITY ARCHITECTURE



DEFENSE LAYERS EXPLAINED

- 1 PERIMETER SECURITY**
WAF, firewalls, DDoS protection, TLS termination, IP allowlists.
- 2 IDENTITY & ACCESS MANAGEMENT**
Strong authentication, SSO, least privilege access, MFA.
- 3 NETWORK SECURITY**
Network segmentation, zero trust between services, restrict traffic with network policies.
- 4 WORKLOAD SECURITY**
Harden containers and pods, scan images, enforce security standards at runtime.
- 5 APPLICATION SECURITY**
Secure coding, validate inputs, manage secrets, protect APIs and endpoints.
- 6 DATA PROTECTION**
Encrypt data, manage keys, limit exposure, classify and minimize data.
- 7 MONITORING & RESPONSE**
Centralized logs, metrics, traces, alerts, and automated response playbooks.



BEST PRACTICES



Apply least privilege access everywhere.



Automate security checks in CI/CD.



Continuously monitor and detect anomalies.



Patch, update, and remediate continuously.



Test and validate your defenses.



Build a security-first culture and mindset.

AUTHENTICATION VS. AUTHORIZATION · IDENTITY AND ACCESS MANAGEMENT · OAUTH 2.0 FUNDAMENTALS



AUTHENTICATION: WHO ARE YOU?

- Verifies identity: username/password, MFA, OAuth2/OIDC login, API key or certificate



AUTHORIZATION: WHAT CAN YOU DO?

- Checks permissions: roles, scopes in an access token, ACLs, resource-level permissions
- First prove who you are, then determine what you can do



HOW IAM WORKS

- Identities (users, services, apps, devices) -> Authentication -> Authorization -> Access
- Continuous monitoring wraps around the entire flow



COMMON IAM CAPABILITIES

- Single sign-on (SSO), multi-factor authentication, RBAC, ABAC
- Privileged access management (PAM), identity federation and social login



CORE ROLES

- Resource owner: the user who owns the data
- Client: the application requesting access on the user's behalf
- Authorization server: authenticates the user and issues access tokens
- Resource server: hosts the protected resources

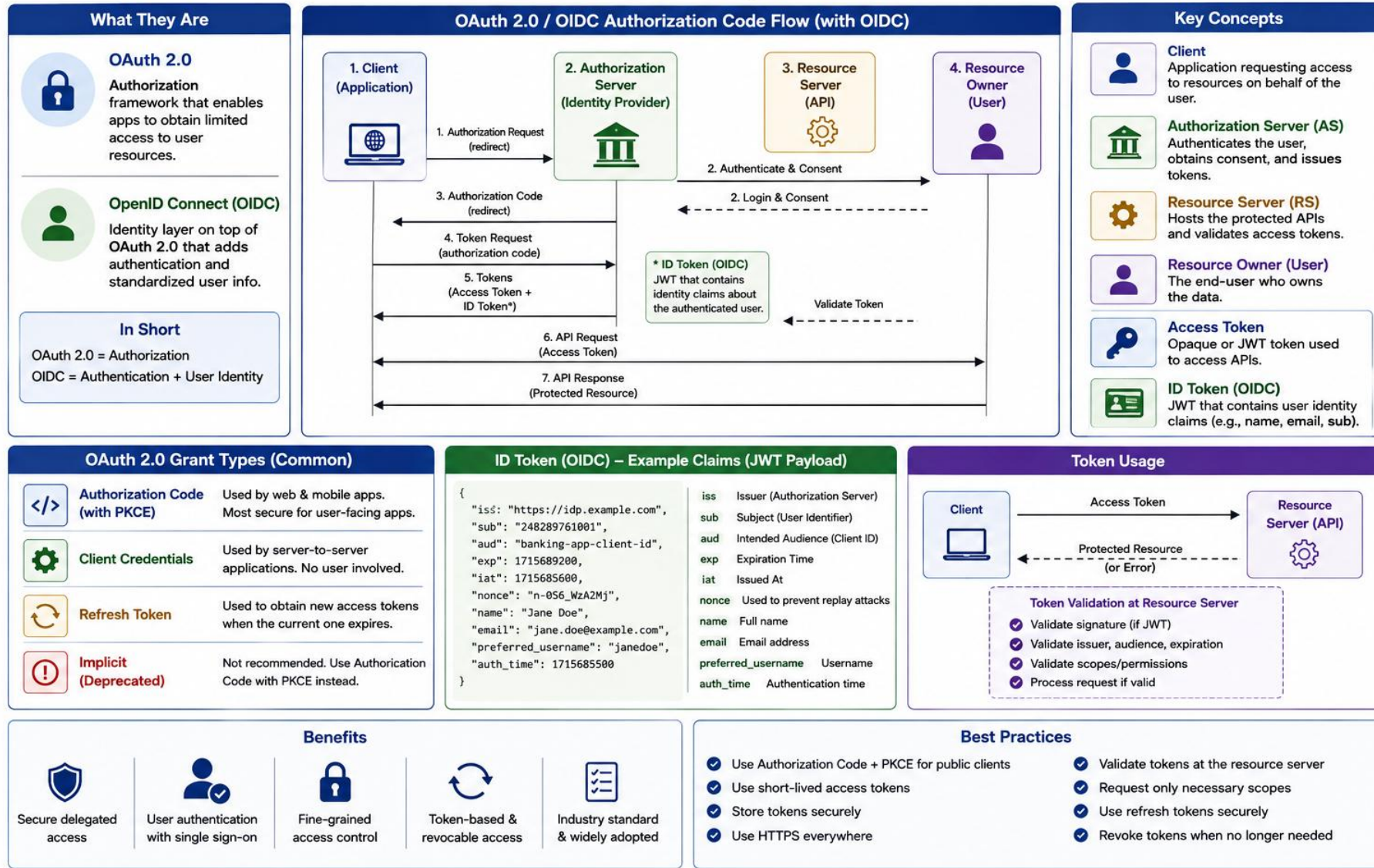


BEST PRACTICES

- Use least-privilege scopes; always use HTTPS; validate tokens on the resource server
- Keep tokens short-lived and never store sensitive data in an access token

KEY TAKEAWAY A customer may prove who they are through login, but authorization is the separate check that decides which accounts they can actually view. • Identities, authentication, authorization, and access flow through one governed framework -- with audit and monitoring wrapped around all of it. • Resource owner, client, authorization server, and resource server are OAuth2's four roles -- each with one clear responsibility.

OAuth 2.0 and OpenID Connect



OPENID CONNECT FUNDAMENTALS · OAUTH ROLES AND AUTHORIZATION FLOWS



TOKENS IN OIDC

- ID Token (JWT): identity information about the authenticated user (sub, name, email)
- Access Token: used to access protected resources (APIs)
- Refresh Token: used to obtain new access tokens without user interaction



KEY BENEFITS

- Single sign-on (SSO) across applications, standardized identity information
- Delegated access with OAuth2, reduces custom authentication implementations



FLOW SELECTION GUIDE

- Web application (server-side) / Mobile / Native: Authorization Code (with PKCE)
- Single-page application (SPA): Authorization Code with PKCE
- First-party/trusted app: Resource Owner Password Credentials
- Microservice / backend-to-backend: Client Credentials

KEY TAKEAWAY The ID token is what OIDC adds: a JWT carrying identity claims like sub, name, and email that OAuth2 alone never provided. • Web and mobile apps use authorization code with PKCE; server-to-server calls use client credentials -- picking the wrong flow weakens security.

AUTHORIZATION CODE FLOW WITH PKCE · CLIENT CREDENTIALS FLOW · ACCESS TOKENS, ID TOKENS, AND REFRESH TOKENS

HOW PKCE WORKS

- 1. Client generates a high-entropy code_verifier
- 2. Client derives code_challenge = BASE64URL(SHA256(code_verifier))
- 3. Authorization request includes the code_challenge
- 4. Token request includes the original code_verifier for validation

HOW IT WORKS

- Client authenticates with the authorization server using its own credentials
- No user involvement -- the client accesses resources as itself
- Common use: microservice-to-microservice calls, scheduled jobs, background services

WHY PKCE MATTERS

- Essential for public clients (SPAs, mobile, native apps) with no client secret
- Now recommended for all OAuth2 clients per RFC 7636

TOKEN COMPARISON

FEATURE	ACCESS TOKEN	ID TOKEN	REFRESH TOKEN
Primary use	Access APIs	Authenticate user	Get new tokens
Sent to API	Yes	No	No
Lifetime	Short	Short	Long

KEY TAKEAWAY A stolen authorization code is useless without the original code_verifier -- that's the entire security value PKCE adds for public clients. • Notification Service authenticates as itself, gets a token, and calls Customer Service -- no end-user session involved anywhere. • Access tokens are short-lived and sent to APIs; ID tokens are never sent to APIs; refresh tokens are long-lived and stored securely by the client.

User-Delegated Access vs Workload Identity

User-Delegated Access

(On-Behalf-Of a User)

An application acts on behalf of a signed-in user, with the user's consent and the user's permissions.

1. User

Signs in (MFA)

2. Client Application

Requests access to APIs

3. Authorization Server (IdP)

Validates token and returns data

4. Resource Server (API)

Validates token and returns data

Authorization Request

Access Token (for user)

Token contains user context and permissions

```
{  "iss": "https://idp.example.com",  "sub": "user123",  "aud": "api://banking-api",  "scope": "accounts.read transactions.initiate",  "roles": ["customer"],  "name": "Jane Doe",  "preferred_username": "jane.doe@example.com"}}
```

Common Use Cases

- Customer viewing account balances
- User initiating a payment
- User managing their profile
- Any action requiring the user's identity and consent

Key Characteristics

- Requires user authentication and consent
- Token represents the user (user context in token)
- Access is limited to the user's permissions
- Subject to user lifecycle and revocation
- Example: Mobile app, Web app calling APIs on behalf of a user

Workload Identity

(Application-to-Application)

An application (workload/service) accesses APIs using its own identity, without a user context.

1. Workload / Service

Daemon, service, or background job

2. Authorization Server (IdP)

Validates token and returns data

3. Resource Server (API)

Validates token and returns data

Token Request (Client Credentials or Federation)

Access Token (for workload)

Token contains workload (client) identity, not a user

```
{  "iss": "https://idp.example.com",  "sub": "spn:banking-backend",  "aud": "api://banking-api",  "scope": "transactions.process",  "roles": ["processor"],  "appid": "banking-backend",  "tid": "tenant-1234"}}
```

Common Use Cases

- Backend services calling other services
- Scheduled jobs / batch processing
- System-to-system integrations
- No end-user involved

Key Characteristics

- No user involved
- Token represents the application/workload
- Access is based on the application's permissions
- Not affected by user sign-in or user changes
- Example: Microservice calling another API, nightly reconciliation job

	At a Glance	User-Delegated Access	Aspect	Workload Identity
			Identity	
	Acts on behalf of	A signed-in user		An application / workload
	Authentication	User authentication (interactive)		Application authentication (non-interactive)
	Token subject (sub)	User ID		Application / Service Principal ID
	Consent	User (and admin) consent required		Admin consent (application permissions)
	Typical Flow	Authorization Code Flow (with OIDC)		Client Credentials Flow or Workload Identity Federation
	Example	Mobile banking app		Transaction processor service

JSON WEB TOKENS · JWT STRUCTURE AND VALIDATION



HOW A JWT IS USED

1. Client logs in with credentials
2. Auth server authenticates and issues a JWT
3. Client sends the JWT in the Authorization header
4. Resource server validates the token and returns a response



VALIDATION CHECKS

- Signature is valid; token is not expired (exp) or used before its valid time (nbf)
- Audience (aud) is valid; issuer (iss) is trusted; subject (sub) is acceptable



KEY POINTS

- JWTs are stateless -- the server does not store session information
- Never store sensitive data in the payload; always use HTTPS



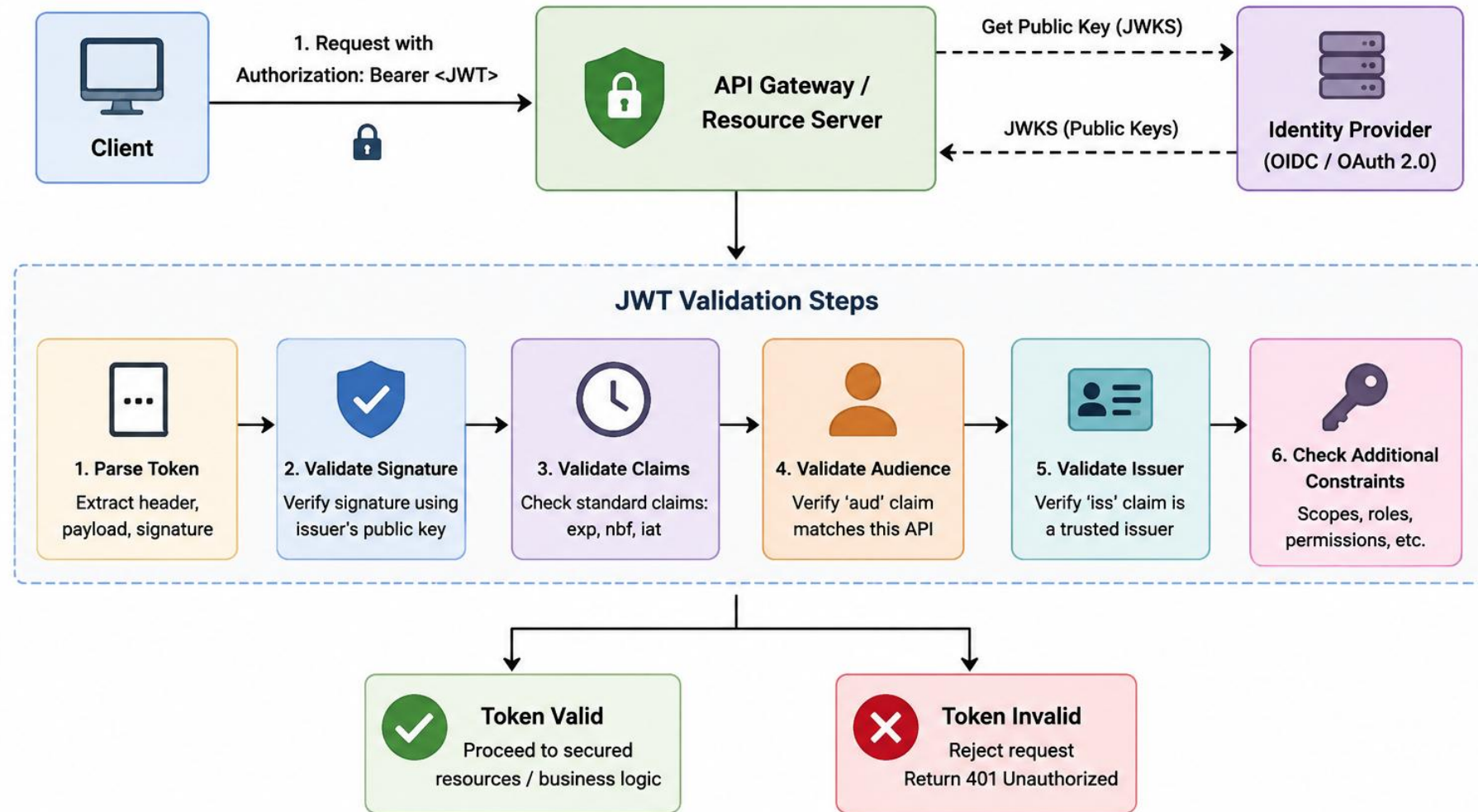
SYMMETRIC VS. ASYMMETRIC SIGNING

- Symmetric (HMAC): HS256/384/512, shared secret key, faster
- Asymmetric (RSA/ECDSA): RS256/ES256, private key signs, public key verifies -- more secure for distributed systems

KEY TAKEAWAY header.payload.signature -- three Base64Url-encoded parts that let a resource server trust claims without a database lookup.

- Split, decode, validate, verify signature, then trust -- five steps stand between an incoming bearer token and an authorized request.

JWT Validation



TOKEN EXPIRATION AND ROTATION · SCOPES, ROLES, AND CLAIMS



COMMON TOKEN LIFETIMES

- Access token: 5-15 minutes ID token: 15-60 minutes
- Refresh token: days to weeks (rotate on each use)



SCOPES VS. ROLES

- Scopes: fine-grained, API-specific, granted to the client application (read:orders, write:orders)
- Roles: broader, organization-specific groupings assigned to the user (USER, MANAGER, ADMIN)



ROTATION BENEFITS AND REUSE DETECTION

- Limits damage if a refresh token is stolen; enables detection of token theft
- Reuse of an old refresh token should invalidate the whole token family and force re-authentication

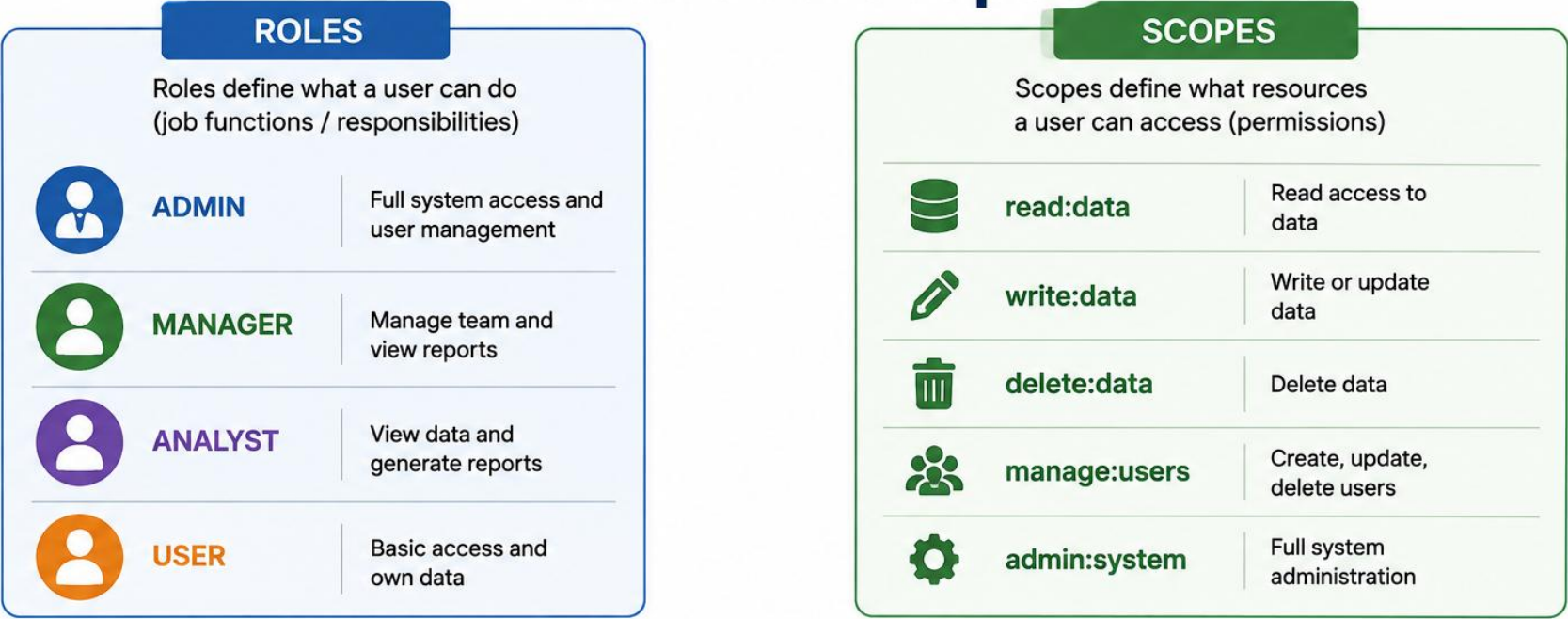


SECURITY NOTE

- Claims are readable by anyone with the token -- never put sensitive data in them
- Use roles to assign permissions; use scopes to limit token capabilities

KEY TAKEAWAY If a rotated refresh token is ever reused, that's a signal of theft -- the entire token family should be revoked immediately. • read:orders is a scope granted to the client; MANAGER is a role assigned to the user -- conflating the two weakens authorization design.

Roles and Scopes



Roles are assigned Scopes



KEYCLOAK ARCHITECTURE · AZURE AD AND MICROSOFT ENTRA ID INTEGRATION · SECURING SPRING BOOT WITH SPRING SECURITY



KEYCLOAK DATA MODEL

- Realm: top-level container for tenants or domains (e.g., 'banking')
- Client: an application or service that uses Keycloak (customer-service, account-service)
- Users, Roles, Groups: identity, permissions, and logical groupings



KEY COMPONENTS

- Authentication services, authorization services, user federation, token services
- Supports OAuth2, OpenID Connect, and SAML 2.0



WHY IT MATTERS

- Single sign-on, enterprise user identities, group-based access, conditional access
- App registrations define scopes, app roles, redirect URLs, and secrets/certificates



SPRING BOOT CONFIGURATION

- spring.security.oauth2.resourceserver.jwt.issuer-uri:
`https://login.microsoftonline.com/{tenant-id}/v2.0`
- Validate issuer, audience, and roles/groups claims from the ID and access



HIGH-LEVEL SECURITY FLOW

- User sends request -> filter chain intercepts -> authenticate -> authorize -> access granted/denied



AUTHORIZATION STRATEGIES

- Roles (hasRole), authorities (hasAuthority), method security (@PreAuthorize), URL security
- Disable CSRF for stateless APIs; use stateless session creation policy

KEY TAKEAWAY Realms, clients, users, roles, and groups form Keycloak's data model -- the same building blocks behind every OAuth2/OIDC deployment. • issuer-uri and jwk-set-uri configuration in Spring Boot are all it takes to validate tokens issued by an enterprise's existing Entra ID tenant. • requestMatchers rules plus oauth2ResourceServer().jwt() turn a Spring Boot service into a token-validating, role-aware protected API in a few lines.

CONFIGURING AN OAUTH2 RESOURCE SERVER

issuer-uri enables OIDC discovery, automatically fetching the JWKS endpoint used to verify every incoming token's signature.



SETUP STEPS

1. Add the spring-boot-starter-oauth2-resource-server dependency
2. Configure `spring.security.oauth2.resourceserver.jwt.issuer-uri`
3. Configure the security filter chain with `authorizeHttpRequests` and `oauth2ResourceServer(oauth2 -> oauth2.jwt())`
4. Call the API with Authorization: Bearer <access_token>

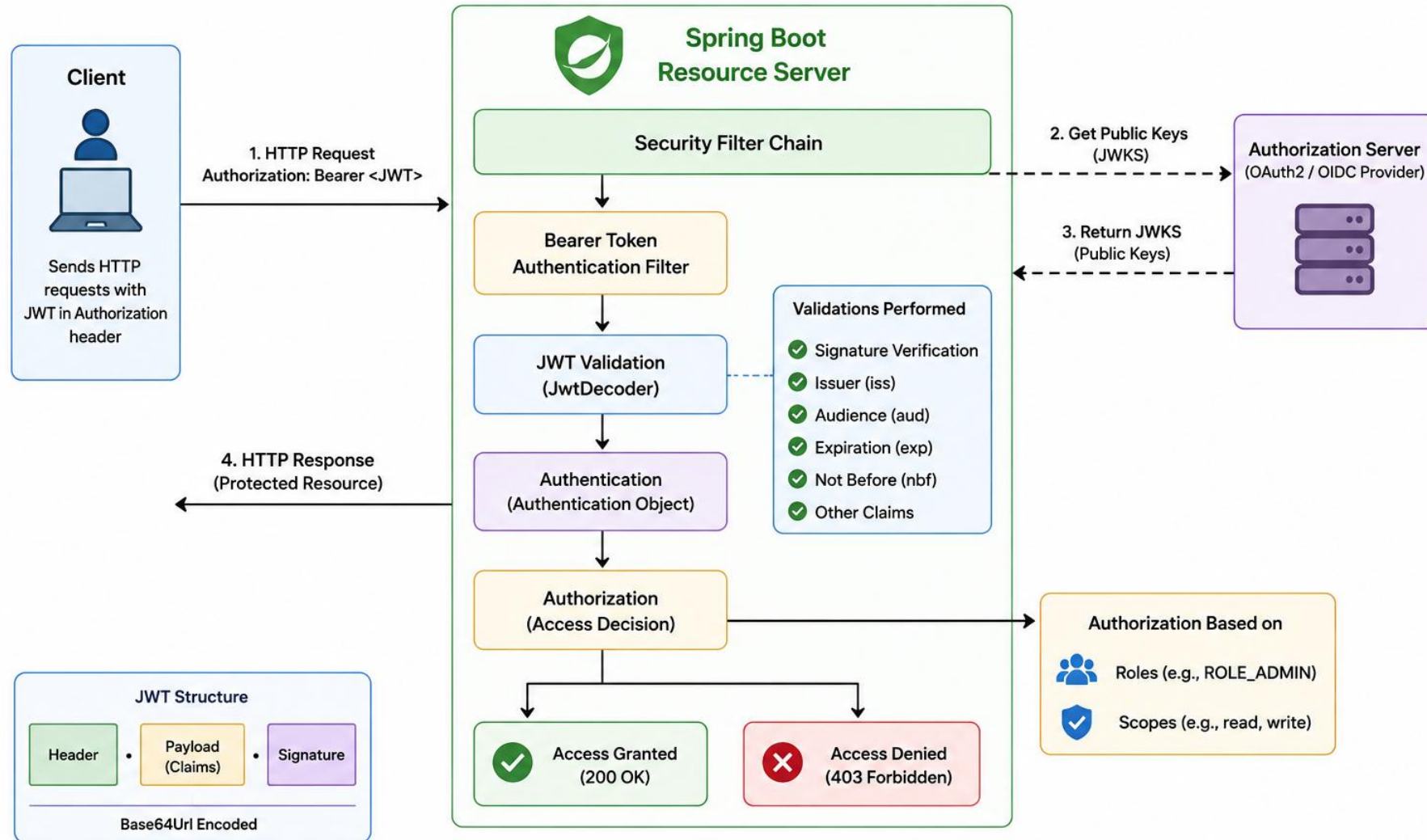


RESPONSE BEHAVIOR

- Invalid or unauthorized tokens return 401 or 403 automatically

KEY TAKEAWAY issuer-uri enables OIDC discovery, automatically fetching the JWKS endpoint used to verify every incoming token's signature.

Spring Security Resource Server



METHOD-LEVEL AUTHORIZATION · ROLE-BASED ACCESS CONTROL · ATTRIBUTE-BASED ACCESS CONTROL



COMMON ANNOTATIONS

- @PreAuthorize: evaluates before method execution, blocks access if the expression is false
- @PostAuthorize: evaluates after execution, useful for validating the result
- @PreFilter / @PostFilter: filter collection arguments or results



BEST PRACTICES

- Use method security for business rules that can't be enforced by URLs alone
- Always combine with authentication and URL-level security for true defense in depth



HOW RBAC WORKS

- Define roles that represent job functions (CUSTOMER, SUPPORT_AGENT, ACCOUNT_MANAGER, ADMIN)
- Assign permissions to each role, then assign roles to users
- Authorization checks a user's role rather than individual permissions



ABAC EVALUATES MULTIPLE ATTRIBUTES

- Subject attributes: role, department, clearance level
- Resource attributes: sensitivity, owner, classification
- Environment attributes: time, location, device, risk score



WHEN TO USE ABAC OVER RBAC

- When access decisions depend on context, not just a static role
- More flexible and fine-grained, at the cost of more complex policy design

KEY TAKEAWAY hasAuthority('order:read') or #id == authentication.name lets one method enforce both a permission check and object ownership together. • A SUPPORT_AGENT role bundles exactly the permissions that job needs; adding a new agent means assigning one role, not a dozen permissions. • A policy like 'branch managers can approve loans under \$50k for their own branch' needs ABAC -- RBAC alone can't express that condition.

SERVICE-TO-SERVICE AUTHENTICATION · API GATEWAY SECURITY · CROSS-ORIGIN RESOURCE SHARING

SERVICE-TO-SERVICE AUTH OPTIONS

- OAuth2 client credentials flow -- each service authenticates with its own client ID/secret
- Mutual TLS (mTLS) -- both sides present certificates to verify identity
- Service accounts and platform-issued identities (e.g., OpenShift service accounts)

WHAT CORS CONTROLS

- Which browser origins may call the API, which HTTP methods, and which headers
- The browser enforces this, not the server -- but the server declares the policy

GATEWAY SECURITY RESPONSIBILITIES

- Token validation and authentication before requests reach any backend service
- Rate limiting, IP allowlisting, and WAF-style request filtering
- TLS termination and consistent security headers across every API

BEST PRACTICE

- Restrict allowed origins explicitly in production; never wildcard when credentials are involved

KEY TAKEAWAY OAuth2 client credentials, mTLS, and service accounts each give one service a verifiable identity when calling another. • Centralizing security at the gateway means individual services don't each have to reimplement the same authentication and throttling logic. • Allowing every origin (*) in production is rarely safe -- CORS should be scoped to the specific frontend domains that legitimately call the API.

CSRF AND STATELESS APIS · TRANSPORT LAYER SECURITY · MUTUAL TLS BETWEEN SERVICES



WHY CSRF DOESN'T APPLY TO STATELESS APIS

- CSRF exploits automatic cookie submission by the browser on cross-site requests
- A stateless API using Authorization: Bearer <token> is not vulnerable the same way
- `http.csrf(csrf -> csrf.disable())` is standard for JWT-secured, stateless REST APIs



WHY TLS IS NON-NEGOTIABLE

- Encrypts traffic between client and server, preventing eavesdropping and tampering
- Certificate validation confirms the server is who it claims to be
- Enforce TLS 1.2+ everywhere, including internal service-to-service calls

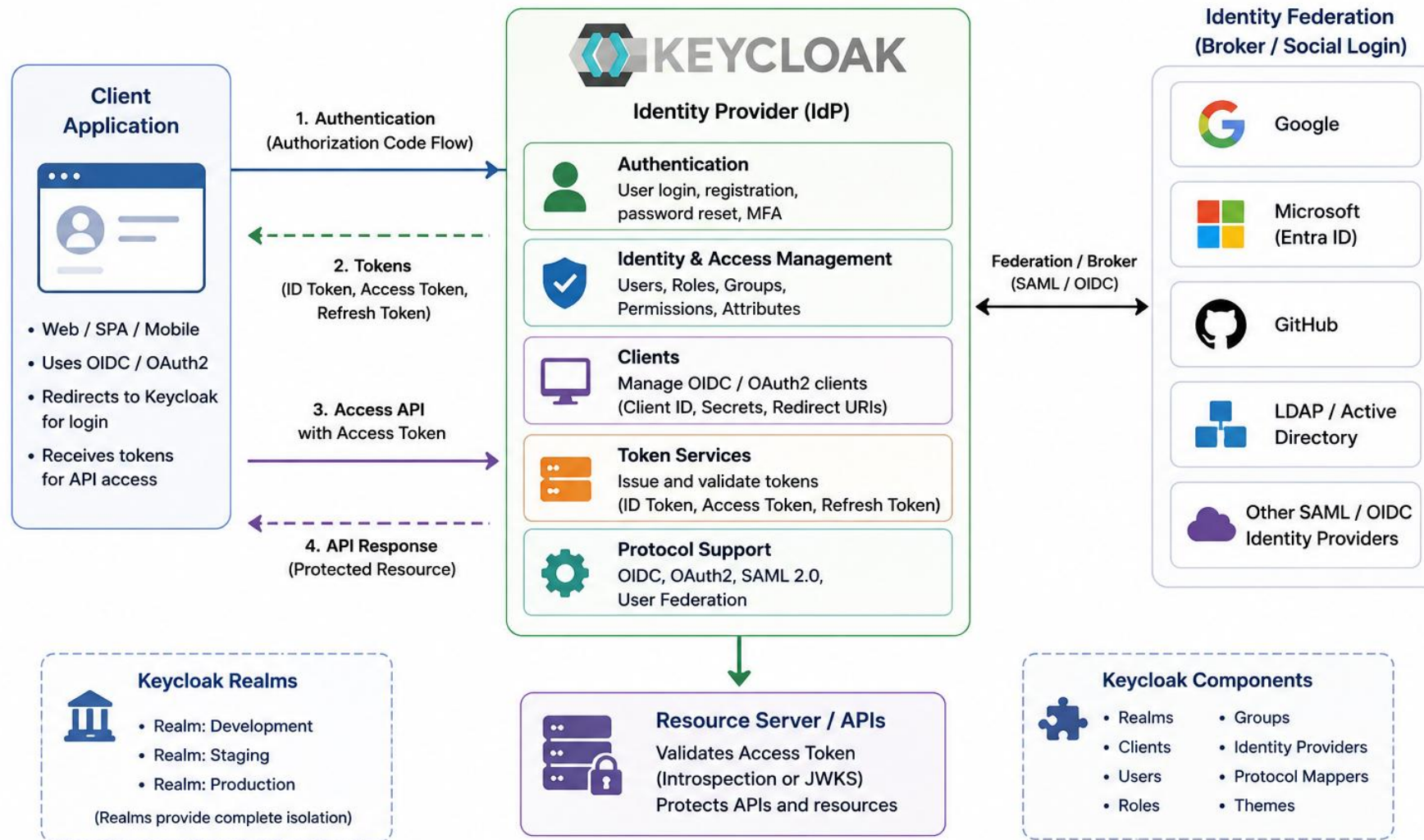


HOW MTLS WORKS

- Client presents a certificate; server presents a certificate; both are validated
- Commonly automated by a service mesh so individual services don't manage certificates directly

KEY TAKEAWAY A bearer token in an Authorization header isn't automatically sent by the browser like a cookie is, which is why stateless APIs don't need CSRF tokens. • Every request in a banking platform -- customer-facing or service-to-service -- must travel over TLS 1.2 or higher, no exceptions. • In a zero-trust internal network, mTLS between services means a compromised network segment still can't impersonate a trusted service.

Keycloak / Client IdP Overview



SECRETS MANAGEMENT FUNDAMENTALS

Database passwords, API keys, and signing keys should be injected at runtime, not baked into the application at build time.



WHAT COUNTS AS A SECRET

- Database passwords, OAuth client secrets, API keys, JWT signing keys, TLS private keys

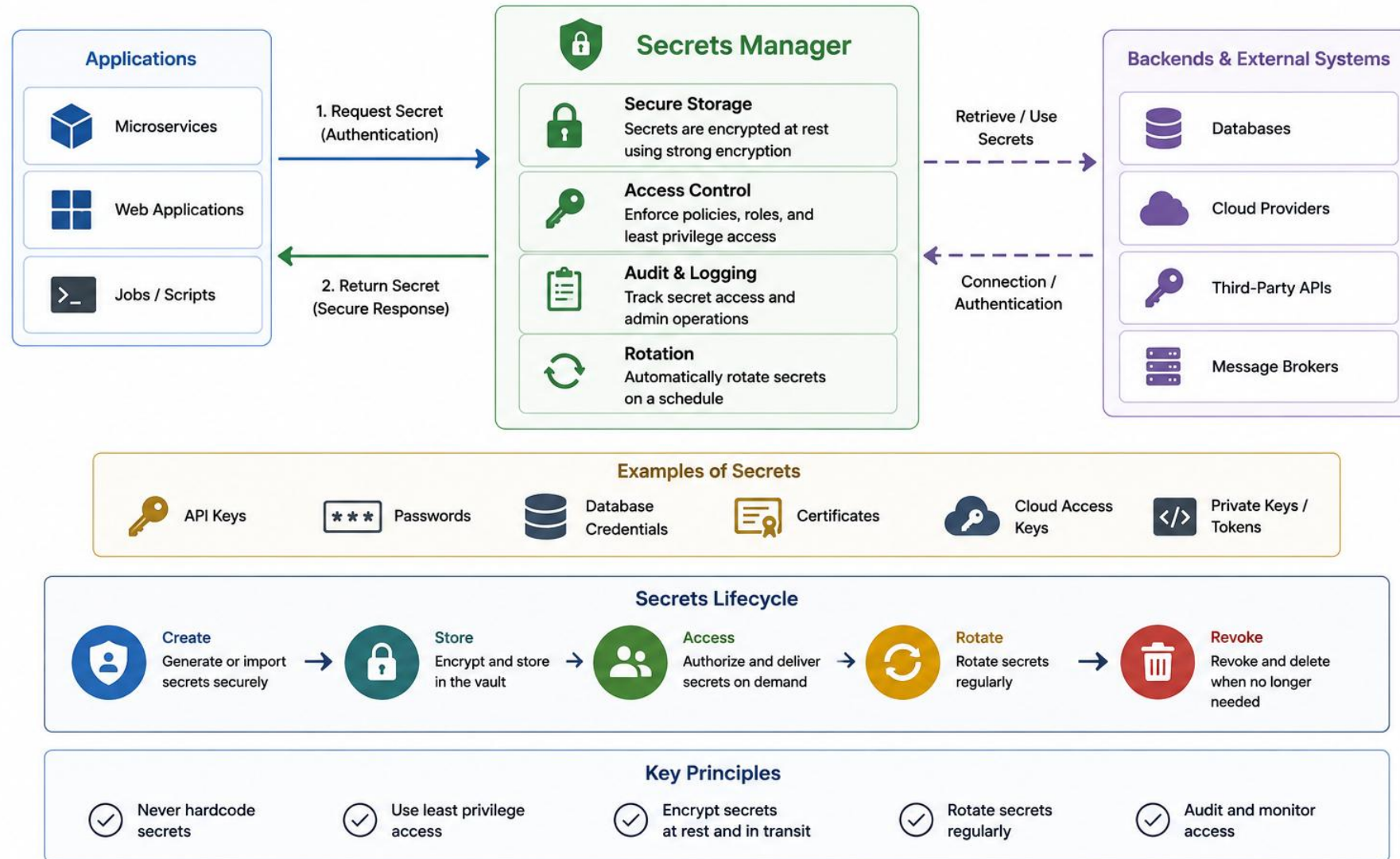


BEST PRACTICES

- Store secrets outside source code, inject at runtime, limit access, rotate regularly
- Use separate secrets per environment and audit secret access

KEY TAKEAWAY Database passwords, API keys, and signing keys should be injected at runtime, not baked into the application at build time.

Secrets Management Basics



OPENSIFT SECRETS · EXTERNAL SECRET STORES AND VAULTS · SECRET ROTATION



OPENSIFT SECRET EXAMPLE

- apiVersion: v1, kind: Secret, type: Opaque
- stringData:
ACCOUNT_DB_PASSWORD: <injected securely>
- Real secret creation should follow enterprise security procedures, not ad hoc values



COMMON EXTERNAL SECRET STORES

- HashiCorp Vault, Azure Key Vault, AWS Secrets Manager, Google Secret Manager
- Provide dynamic secrets, fine-grained access policies, and centralized audit logging



WHEN TO ROTATE SECRETS

- Periodically, after suspected exposure, when team members change roles
- When service accounts are replaced, or after a security incident

KEY TAKEAWAY A Secret of type Opaque, mounted as an environment variable, keeps ACCOUNT_DB_PASSWORD out of application.yml entirely.

- An external vault can issue short-lived, on-demand database credentials instead of one static password shared across every service instance.
- Rotating periodically, after a suspected exposure, or when a team member changes roles limits the blast radius of any single leaked credential.

API INPUT VALIDATION · OWASP API SECURITY RISKS · RATE LIMITING AND ABUSE PREVENTION



WHAT TO VALIDATE

- Path variables, query parameters, request bodies, headers where relevant, file uploads



TOP OWASP API RISKS

- Broken object level authorization, broken authentication, broken object property level authorization
- Unrestricted resource consumption, broken function level authorization
- Security misconfiguration, improper inventory management



BEST PRACTICES

- Validate all inputs on the server side; use allowlists, not denylists
- Use parameterized queries; validate files by type, size, and content



WHERE AND WHY

- Apply limits close to the entry point: API gateway, load balancer, or service mesh
- Use different limits for different clients or operations (login vs. balance lookup vs. payment)
- Return clear 429 responses and monitor rate-limit rejections

KEY TAKEAWAY Unvalidated input is the root cause behind injection attacks, broken object authorization, and countless other API vulnerabilities. • Customer 101 changing a URL to /accounts/5002 and getting another customer's data is exactly the failure OWASP's list is built around. • 429 Too Many Requests is the standard signal that tells a well-behaved client to back off -- applied at the gateway, closest to the entry point.

Data Classification, Minimization, and Log Redaction

1. Data Classification

Classify data based on sensitivity and impact

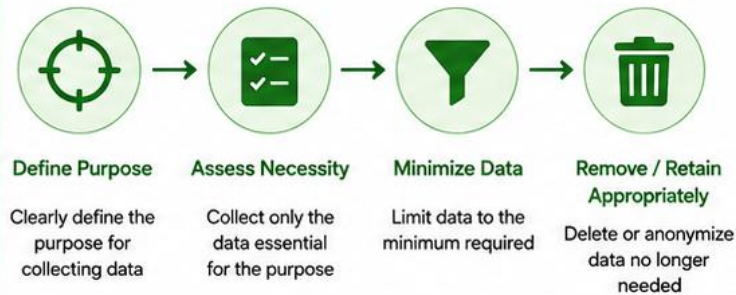
 Critical	Highest sensitivity Highest impact	SSN, Payment Details, Health Records
 Confidential	High sensitivity High impact	PII, Credentials, Personal Data
 Internal	Moderate sensitivity Moderate impact	Employee Info, Business Data
 Public	Low sensitivity Low impact	Marketing Info, Public Content

Classification Guidelines

-  Identify the type of data
-  Assess sensitivity and business impact
-  Assign appropriate classification level
-  Review and update classifications regularly

2. Data Minimization

Collect, process, and retain only what is necessary



Minimization Principles

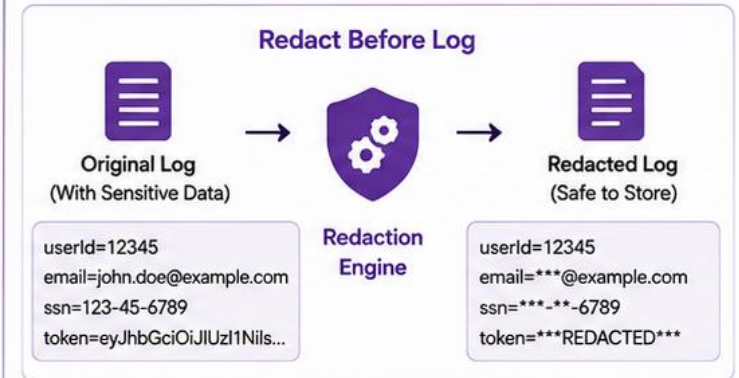
- ✓ Collect only necessary data
- ✓ Avoid collecting data "just in case"
- ✓ Use aggregated or anonymized data when possible
- ✓ Set clear retention periods
- ✓ Regularly review and purge unnecessary data

Benefits



3. Log Redaction

Prevent sensitive data from being written to logs



What to Redact



Redaction Best Practices

- ✓ Maintain a list of sensitive data patterns and fields
- ✓ Redact at the application level, not just in log aggregation
- ✓ Use consistent masking (e.g., *** or partial masking)
- ✓ Never log secrets, tokens, or full raw payloads
- ✓ Regularly test logs to ensure no sensitive data leaks

SECURITY LOGGING AND AUDIT TRAILS · DATA ENCRYPTION AT REST AND IN TRANSIT · PERSONALLY IDENTIFIABLE INFORMATION



WHAT TO LOG

- Failed login attempts, access denied events, account updates
- Payment creation, admin operations, secret rotation, high-risk AI tool calls



TWO ENCRYPTION DOMAINS

- In transit: TLS 1.2+ for all client and service-to-service traffic
- At rest: encrypted databases, encrypted disks, and encrypted backups
- Use secure key management -- never hardcode encryption keys



WHAT NEVER TO LOG

- Passwords, tokens, full account numbers, secret values, sensitive personal data



PROTECTING PII

- Minimize collection of sensitive data to only what's necessary
- Mask, tokenize, or pseudonymize PII when full values aren't required
- Never log PII, and encrypt it both at rest and in transit

KEY TAKEAWAY Audit logs answer 'who did what, when' after an incident -- but only if they were captured consistently before the incident happened. • TLS protects data in transit; database and disk-level encryption protects data at rest -- a complete strategy needs both, not just one. • Names, addresses, SSNs, and account numbers are exactly the data a banking platform must protect most carefully across every system that touches it.

SECURITY AND COMPLIANCE CONTROLS · THREAT MODELING MICROSERVICES · ZERO-TRUST ARCHITECTURE



WHY COMPLIANCE CONTROLS MATTER

- Regulatory and contractual obligations (e.g., GDPR, HIPAA-style requirements) shape required controls
- Controls must be documented, testable, and auditable -- not just implemented



THE THREAT MODELING PROCESS

1. Define scope
2. Create architecture view
3. Identify threats (STRIDE)
4. Assess risks
5. Mitigate & validate
6. Monitor & iterate



BEST PRACTICES

- Model early and often; include automated checks in CI/CD pipelines
- Prioritize risks based on business impact; validate controls with testing (SAST/DAST, pen tests)



ZERO-TRUST PRINCIPLES

- Verify explicitly, use least-privilege access, assume breach
- Reduce attack surface, continuously monitor and re-validate



ZERO TRUST IN MICROSERVICES

- mTLS between services, fine-grained RBAC/ABAC authorization
- API gateway with policy enforcement, observability across traces, logs, and metrics

KEY TAKEAWAY Compliance frameworks translate 'be secure' into specific, auditable controls a banking platform must demonstrably satisfy. • STRIDE -- spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege -- gives every threat a name and a category. • Never trust, always verify, least privilege, assume breach -- four principles that replace perimeter-based security with continuous verification.

SECURE CODING CHECKLIST · INCIDENT RESPONSE CONSIDERATIONS

CHECKLIST AREAS

1. Input validation
2. Authentication & authorization
3. Data protection
4. Error handling & logging
5. Dependencies & components
6. Session management
7. Communication security
8. Code quality & best practices

THE FIVE-STAGE LIFECYCLE

1. Preparation
2. Detection & analysis
3. Containment
4. Eradication & recovery
5. Lessons learned

REMEMBER

- Security is a shared responsibility -- build security in, not on

POST-INCIDENT REVIEW QUESTIONS

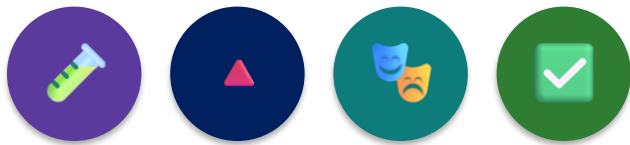
- What happened, why did it happen, and what was the impact?
- What worked, what didn't, and what controls need improvement?

KEY TAKEAWAY Input validation, auth, data protection, error handling, dependencies, sessions, communication, and code quality -- one checklist covers them all. • Speed, coordination, and clear communication separate a minor security event from a major breach -- and none of that works without a plan in place first.

MODULE 8

Testing Spring Boot Microservices

Test at the right layer: unit, slice, contract, and Testcontainers so CI proves the same paths
OpenShift will run.



THE MICROSERVICES TESTING STRATEGY · TESTING PYRAMID VS. TESTING TROPHY · UNIT TESTING WITH JUNIT 5



WHY A STRATEGY MATTERS

- Microservices increase the number of components, interactions, and failure points
- A layered testing strategy reduces risk and optimizes feedback speed
- Fast feedback at lower levels, confidence at higher levels



TESTING PRINCIPLES

- Focus on behavior, not implementation details; test in isolation with clear scope
- Shift left to find defects early; automate for repeatable, reliable results



TESTING PYRAMID

- Classic model with a wide base of unit tests (70-80% unit, 15-20% integration, 5-10% contract/E2E)
- Best for stable teams, well-defined contracts, and systems needing fast CI/CD feedback



TESTING TROPHY

- More emphasis on higher-level tests (20-30% unit, 20-30% integration, 20-30% contract, 30-40% E2E)
- Best for new/rapidly changing products where user value matters more than internals



KEY JUNIT 5 FEATURES

- Modular architecture built on the Jupiter API, extensible
- Simple annotations: @Test, @DisplayName, @BeforeEach, @AfterEach, @Nested
- Powerful assertions, assumptions for conditional tests, dynamic and parameterized tests



BEST PRACTICES

- Test one behavior at a time; keep tests small, focused, and fast
- Use meaningful test and method names; avoid test dependencies and side effects

KEY TAKEAWAY Unit, slice, integration, contract, and end-to-end tests each answer a different question at a different cost. • Stable, contract-driven systems suit the pyramid's unit-heavy mix; fast-changing products suit the trophy's integration-heavy mix. • @Test, @DisplayName, assertions, and parameterized tests turn one behavior into one clear, fast, repeatable test case.

Unit Testing with JUnit / Mockito



JUnit (Testing Framework)



Write tests

Create test methods for each unit of code



Run tests

Execute tests and get results



Assertions

Verify expected outcomes



Reports

View test results and coverage



Mockito (Mocking Framework)



Create Mocks

Mock dependencies and external collaborators



Define Behavior

Stub method calls and define return values / exceptions



Verify Interactions

Verify method calls and interactions



Isolate Unit

Test unit in isolation from external dependencies



Example

Service: UserService

```
public class UserService {  
    private final UserRepository repo;  
    public UserService(UserRepository repo) {  
        this.repo = repo;  
    }  
  
    public User getById(Long id) {  
        return repo.findById(id)  
            .orElseThrow(() -> new NotFoundException(id));  
    }  
}
```

Test: UserServiceTest

```
@ExtendWith(MockitoExtension.class)  
class UserServiceTest {  
    @Mock  
    private UserRepository repo;  
  
    @InjectMocks  
    private UserService service;  
  
    @Test  
    void getById_WhenExists_ReturnsUser() {  
        User user = new User(1L, "John");  
        when(repo.findById(1L)).thenReturn(Optional.of(user));  
  
        User result = service.getById(1L);  
        assertNotNull(result);  
        assertEquals("John", result.getName());  
        verify(repo).findById(1L);  
    }  
}
```

Common Annotations

JUnit 5

@Test

Marks a test method

@BeforeEach

Runs before each test

@AfterEach

Runs after each test

@BeforeAll

Runs once before all tests

@AfterAll

Runs once after all tests

@DisplayName

Custom display name

Mockito

@Mock

Creates a mock

@InjectMocks

Injects mocks into the object

@Spy

Creates a spy (partial mock)

when(...).thenReturn(...)

Stubbing method behavior

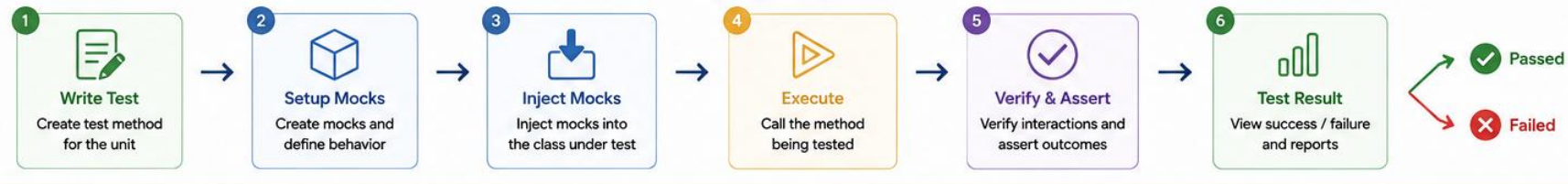
verify(mock).method()

Verifies interactions

ArgumentMatchers

any(), eq(), isNull(), etc.

Unit Test Flow



MOCKING DEPENDENCIES WITH MOCKITO · TESTING CONTROLLERS WITH MOCKMVC

· TESTING JSON REQUESTS AND RESPONSES



KEY MOCKITO FEATURES

- Create mocks easily for interfaces, classes, or concrete dependencies
- Stub behavior: define return values, exceptions, or custom answers
- Verify interactions: confirm methods were called with expected arguments and counts



BEST PRACTICES

- Mock only external dependencies, not the class under test
- Verify interactions, but don't test implementation details -- avoid over-mocking



KEY BENEFITS

- Fast and isolated -- no need to start the application context or embedded server
- Focus on the web layer: request mappings, validations, status codes, responses
- Rich assertions for JSON payloads, headers, and view names



BEST PRACTICES

- Use `@WebMvcTest(Controller.class)` to load only web layer components
- Mock service/repository dependencies with `@MockBean`; cover success, validation, not-found, and security scenarios



WHY IT MATTERS

- Ensures your API contract (request/response) is correct and stable
- Detects serialization, validation, and mapping issues quickly



KEY JSONPATH ASSERTIONS

- `$.field` for exact value, `$.items[*].id` for array values, `$.items.length()` for size
- Always set `contentType`, validate both happy path and error responses

KEY TAKEAWAY `@Mock` creates the fake, `@InjectMocks` wires it in, `when(...).thenReturn(...)` stubs behavior, and `verify(...)` confirms the interaction happened. • `@WebMvcTest` loads only the web layer, `@MockBean` fakes the service, and `mockMvc.perform()` drives a real request through the controller. • Asserting only the status code is a common pitfall; the real API contract lives in the JSON payload's fields, types, and nesting.

Spring Boot Slice Tests

Test a specific layer in isolation with focused application context



OVERVIEW



Slice tests load only a small part of the Spring application context needed for a specific layer (web, data, etc.). They are fast, focused, and ideal for unit-style tests with Spring support.

WHY USE SLICE TESTS?



Fast Execution

Loads minimal context → faster feedback.



Focused Testing

Test one layer at a time (web, data, JSON, etc.).



Less Configuration

Auto-configures only the components you need.



Better Isolation

Mock external dependencies easily.



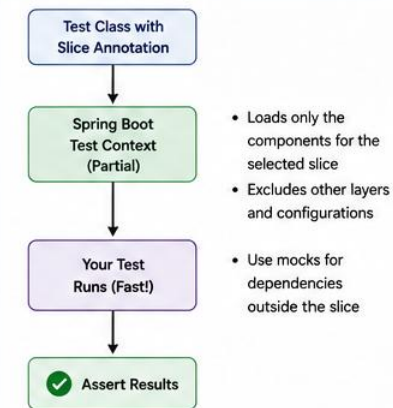
Improved Developer Productivity

Write more tests with less overhead.

COMMON SPRING BOOT SLICE ANNOTATIONS

Annotation	Focus	Loads	Use For
<code>@WebMvcTest</code>	Web Layer (Controllers)	MVC components, Controller, Jackson, Validation, MockMvc	Testing REST controllers, request mapping, JSON, validation
<code>@DataJpaTest</code>	JPA Layer (Repositories)	JPA repositories, EntityManager, H2 (embedded), JPA config	Testing Spring Data JPA repositories and queries
<code>@JsonTest</code>	JSON Layer (Serialization)	Jackson ObjectMapper, JSON components only	Testing JSON serialization/deserialization
<code>@WebFluxTest</code>	WebFlux Layer (Controllers)	WebFlux controllers, codecs, WebTestClient	Testing reactive controllers
<code>@JdbcTest</code>	JDBC Layer	JDBC components, DataSource, JdbcTemplate	Testing JDBC queries
<code>@DataMongoTest</code>	MongoDB Layer (Repositories)	Mongo repositories, MongoTemplate, Embedded Mongo	Testing Spring Data MongoDB repositories

HOW SLICE TESTS WORK



EXAMPLE: @WebMvcTest FOR CONTROLLER TESTING

```
1 @WebMvcTest(UserController.class)
2 class UserControllerTest {
3
4     @Autowired
4     private MockMvc mockMvc;
5
6     @MockBean
6     private UserService userService;
7
8     @Test
9     void shouldReturnUser_whenUserExists() throws Exception {
10         User user = new User(1L, "John Doe", "john@example.com");
11         when(userService.findById(1L)).thenReturn(Optional.of(user));
12
13         mockMvc.perform(get("/api/users/1").accept(MediaType.APPLICATION_JSON))
14             .andExpect(status().isOk())
15             .andExpect(jsonPath("$.name").value("John Doe"))
16             .andExpect(jsonPath("$.email").value("john@example.com"));
17     }
18 }
```

OTHER SLICE EXAMPLES

```
// Test JPA Repository
@DataJpaTest
class UserRepositoryTest {
    @Autowired private EntityManager em;
    @Autowired private UserRepository repo;
    // tests...
}

// Test JSON Serialization
@JsonTest
class UserJsonTest {
    @Autowired private ObjectMapper objectMapper;
    // tests...
}
```

TIPS & BEST PRACTICES

- ✓ Use slice tests for fast and focused feedback.
- ✓ Prefer slice tests over `@SpringBootTest` for unit-style tests.
- ✓ Mock external dependencies with `@MockBean`.
- ✓ Assert both status codes and JSON content.
- ✓ Name tests clearly to describe behavior.
- ✓ Combine slices with integration tests for full coverage.
- ✓ Use `Testcontainers` with `@DataJpaTest` when needed.

WHEN TO USE WHAT?

Use Slice Tests When

- ✓ Testing a single layer
- ✓ Need fast execution
- ✓ Mocking dependencies

Use @SpringBootTest When

- ✓ Cross-layer integration
- ✓ Full context is required
- ✓ End-to-end scenarios



TIP

Slice tests give you the best of both worlds: the power of Spring Boot testing with the speed of unit tests.



Write More Tests. Get Faster Feedback. Ship Better Code.

Built with Spring Boot



REPOSITORY TESTING WITH DATAJPA TEST · INTEGRATION TESTING WITH SPRINGBOOTTEST



WHAT @DATAJPA TEST PROVIDES

- Configures an in-memory database (H2 by default), scans @Entity and @Repository
- Configures JPA, Hibernate, DataSource; transactional tests roll back after each test
- Does NOT load the web layer, services, or other slices



WHY USE @SPRINGBOOTTEST

- End-to-end validation of the entire application stack; uses real configuration and beans
- Tests multiple layers together: controllers, services, repositories, infrastructure
- Works with embedded databases or Testcontainers for real services



BEST PRACTICES

- Use TestEntityManager for fine-grained JPA control; clear the persistence context to test real state
- Combine @DataJpaTest with Testcontainers for higher-fidelity database engines

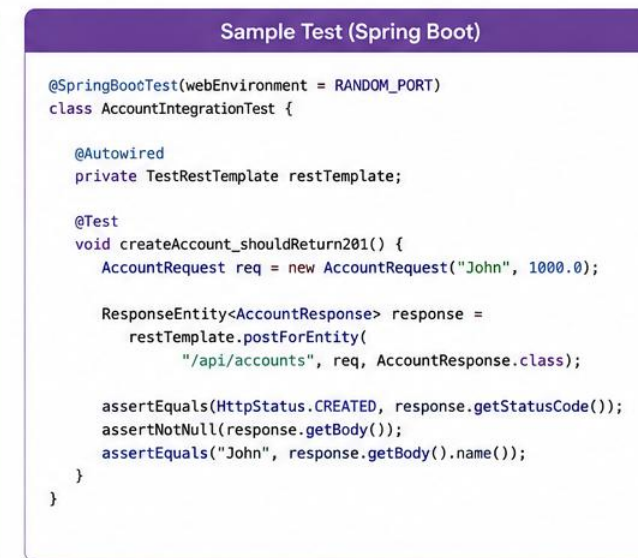
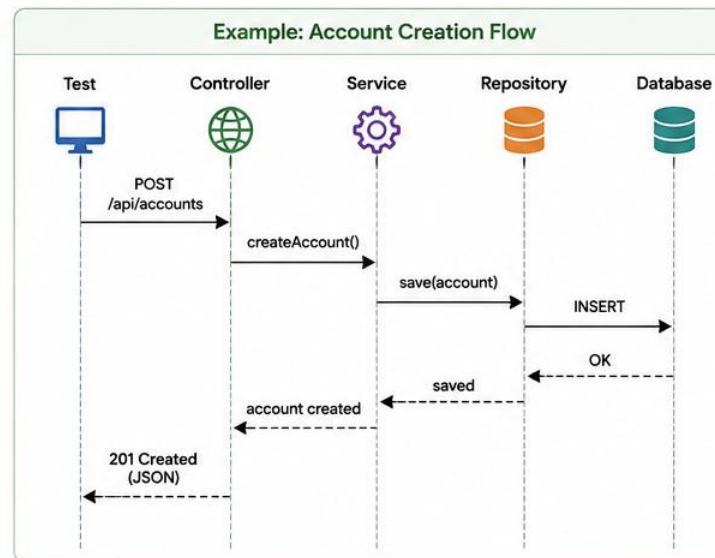
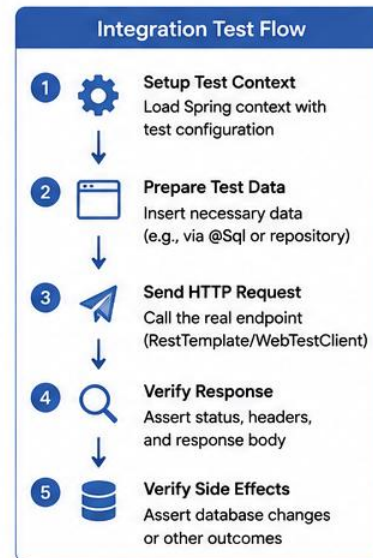
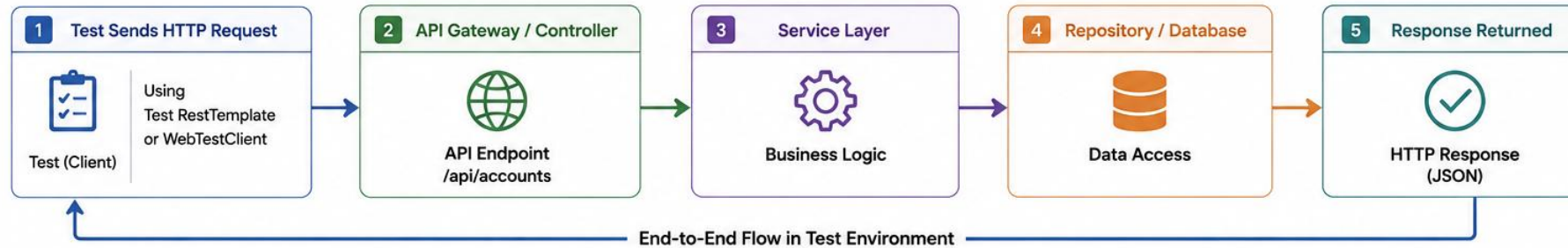


BEST PRACTICES

- Use @SpringBootTest sparingly; prefer slice tests for unit scope
- Use @ActiveProfiles("test") for test-specific config; isolate and clean up test data

KEY TAKEAWAY TestEntityManager gives fine-grained control to persist, flush, and clear the persistence context so queries hit the real database, not a cache. • Controllers, services, repositories, and infrastructure all run together -- catching integration issues a slice test can never see.

One Guided Integration Test



What This Test Verifies



Full request flow
Controller → Service →
Repository → Database



Real components
Tests wiring, configuration,
transactions



Contracts
Validates request/response
formats and status codes



Side effects
Ensures data is persisted
as expected



Confidence
Catches issues unit tests
may miss

Best Practices

- ✓ Use @SpringBootTest with RANDOM_PORT
- ✓ Use real database (Testcontainers or embedded)
- ✓ Reset data between tests
- ✓ Keep tests independent and deterministic

TESTING EXTERNAL SERVICE CALLS · TESTCONTAINERS OVERVIEW



TECHNIQUES TO TEST EXTERNAL CALLS

- Mock with @MockBean: replace the HTTP client bean and stub responses
- MockServer / WireMock: a lightweight HTTP server returning stubbed JSON responses
- Testcontainers + real service: run the real external service in a container for higher fidelity



KEY BENEFITS

- Real dependencies -- test against the real thing, not mocks
- Isolated and reproducible: clean containers for every test run
- Works locally and in CI, with automatic lifecycle management



BEST PRACTICES

- Test both happy path and failure scenarios: timeouts, 4xx/5xx errors, malformed responses
- Keep tests deterministic and independent

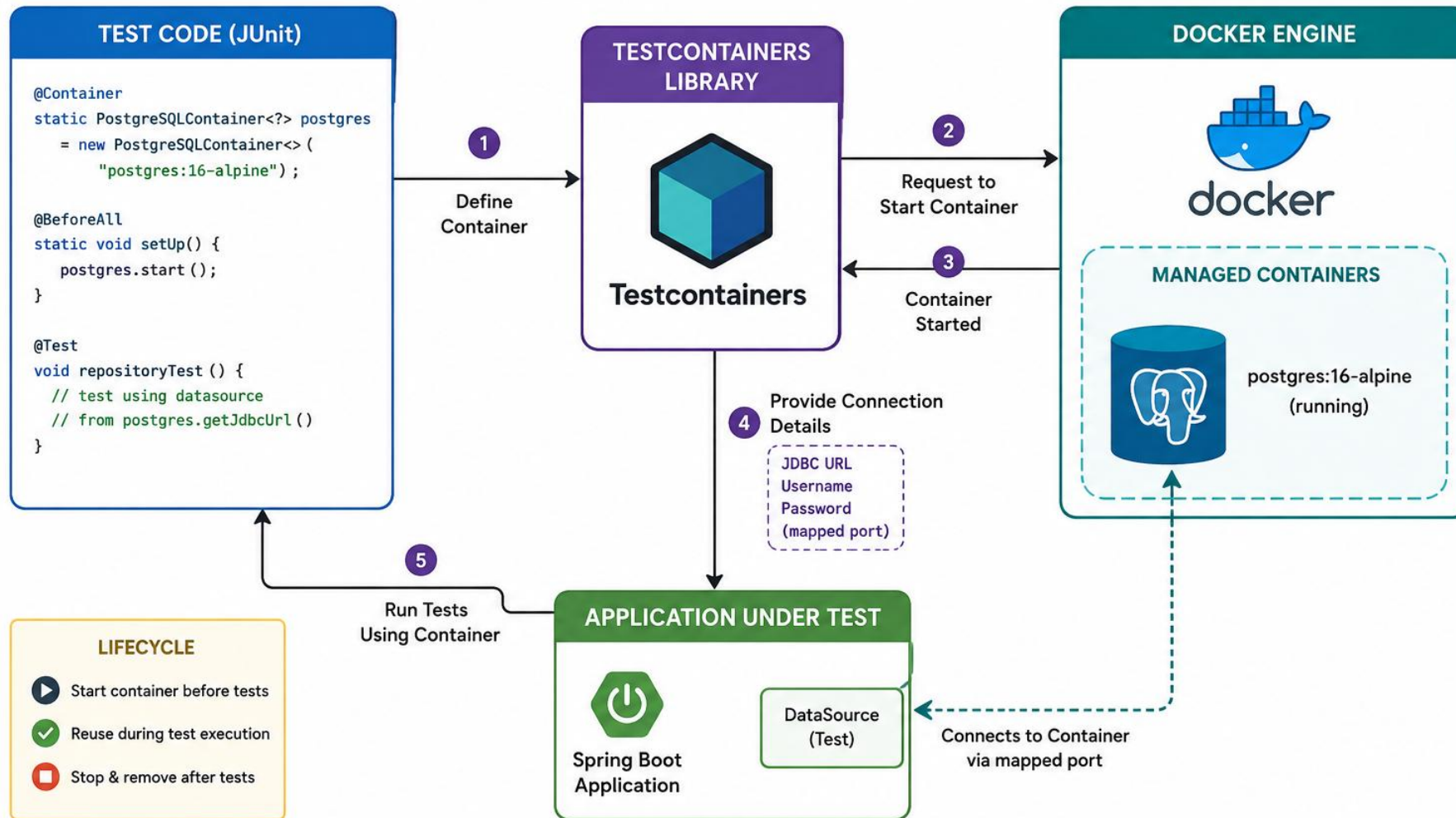


COMMONLY USED MODULES

- PostgreSQL, MySQL, Redis, Kafka, RabbitMQ, Elasticsearch modules, and many more

KEY TAKEAWAY Mocking with @MockBean, running MockServer/WireMock, or spinning up the real service in a container each avoid flaky, costly, real external calls. • Throwaway, isolated, and reproducible environments replace mocks with the real infrastructure your service actually depends on.

TESTCONTAINERS AWARENESS (IF ENVIRONMENT PERMITS)



Database Integration Tests with Testcontainers

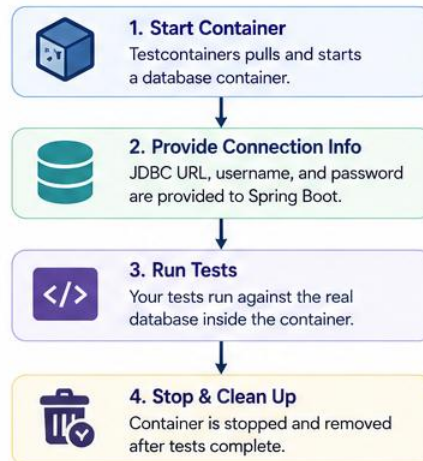
Test your persistence layer against a real database in an isolated container



WHY USE TESTCONTAINERS FOR DATABASE TESTS?

- ✓ Uses a real database – not an in-memory substitute
- ✓ Reproducible and isolated – clean DB for every test run
- ✓ Works locally and in CI – same behavior everywhere
- ✓ Matches production – same engine, version, and config
- ✓ Catches integration issues early – SQL, mappings, constraints

HOW IT WORKS (FLOW)



EXAMPLE (SPRING BOOT + JPA + TESTCONTAINERS)

```
@Testcontainers
@DataJpaTest
class UserRepositoryIT {

    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine")
            .withDatabaseName("testdb")
            .withUsername("test")
            .withPassword("test");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
        registry.add("spring.datasource.driver-class-name", postgres::getDriverClassName);
    }

    @Autowired
    private UserRepository userRepository;

    @Test
    void shouldSaveAndFindUser() {
        User user = new User(null, "Alice", "alice@example.com");
        userRepository.save(user);

        Optional<User> found = userRepository.findByEmail("alice@example.com");
        assertTrue(found.isPresent());
        assertEquals("Alice", found.get().getName());
    }
}
```

WHAT HAPPENS?



CONFIGURATION OPTIONS

- Use different DB versions/images
- Set init scripts:

```
.withInitScript("schema.sql")
.withInitScript("data.sql")
```
- Reuse containers for faster builds:

```
testcontainers.reuse.enable=true
```

BENEFITS

- ✓ Real database behavior (constraints, indexes, transactions, isolation)
- ✓ Validates SQL and JPA mappings
- ✓ Detects dialect-specific issues
- ✓ Eliminates "it works on my machine" problems

BEST PRACTICES

- Use `@DataJpaTest` for repository slice tests
- Use `@SpringBootTest` for full-stack DB tests
- Keep containers lightweight (alpine images)
- Use init scripts for schema/migrations if needed
- Avoid hardcoded ports – let Testcontainers manage
- Reuse containers locally to speed up test runs



KEY TAKEAWAY

Testcontainers brings real infrastructure to your tests. For database integration tests, it provides confidence that your data layer works exactly as expected in production.



RESULT: High-quality, reliable tests that catch integration issues early and reduce production defects.

MESSAGING INTEGRATION TESTS · CONTRACT TESTING FUNDAMENTALS



WHY USE TESTCONTAINERS FOR MESSAGING

- Tests run against a real broker (Kafka, RabbitMQ, etc.), reproducible and isolated
- Validates message serialization/deserialization and consumer acknowledgments
- Catches configuration and connectivity issues early



BEST PRACTICES

- Inject connection info via @DynamicPropertySource; use unique topics/queues per test
- Use Awaitility for async verifications; keep tests isolated and idempotent



HOW CONTRACT TESTING WORKS

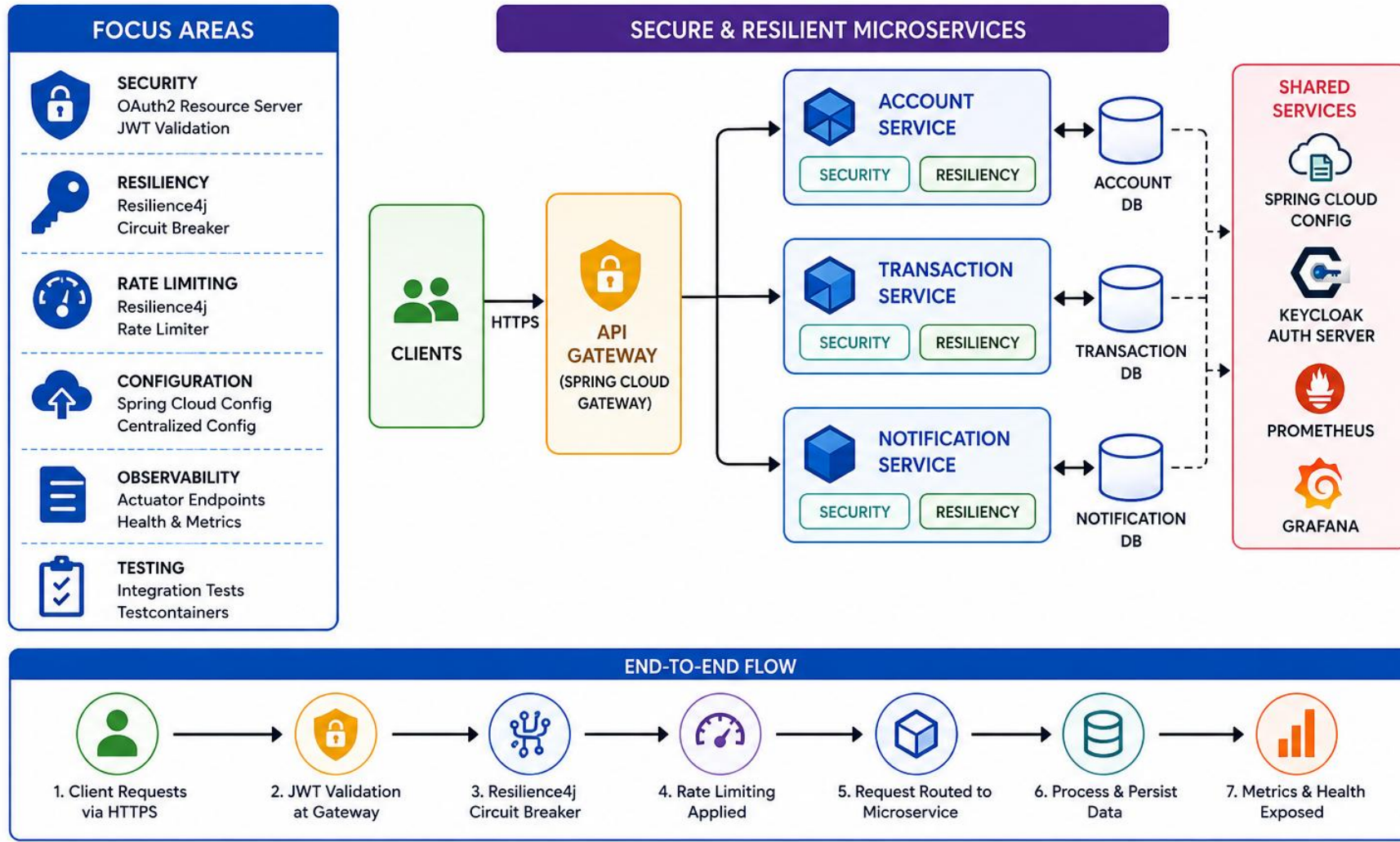
1. Consumer creates a contract defining expected requests and responses
2. The contract (Pact file) is published to a contract repository
3. Provider runs contract tests to verify it meets all consumer expectations
4. If verification passes, the provider can be deployed with confidence

KEY TAKEAWAY Kafka or RabbitMQ running in Testcontainers lets a test publish a real message and assert the consumer actually reacted to it.

- The consumer defines expected requests and responses; the provider verifies it can satisfy that same contract -- both sides trust one shared Pact file.

LAB 3 OVERVIEW

SECURE AND RESILIENT SERVICES



CONSUMER-DRIVEN CONTRACT TESTING

A Pact Broker becomes the shared source of truth, letting the consumer and provider teams release independently without breaking each other.



WHY IT MATTERS

- Reduces integration failures by catching breaking changes before deployment
- Enables independent development -- consumer and provider work in parallel
- Provides living documentation that's always up to date with real usage

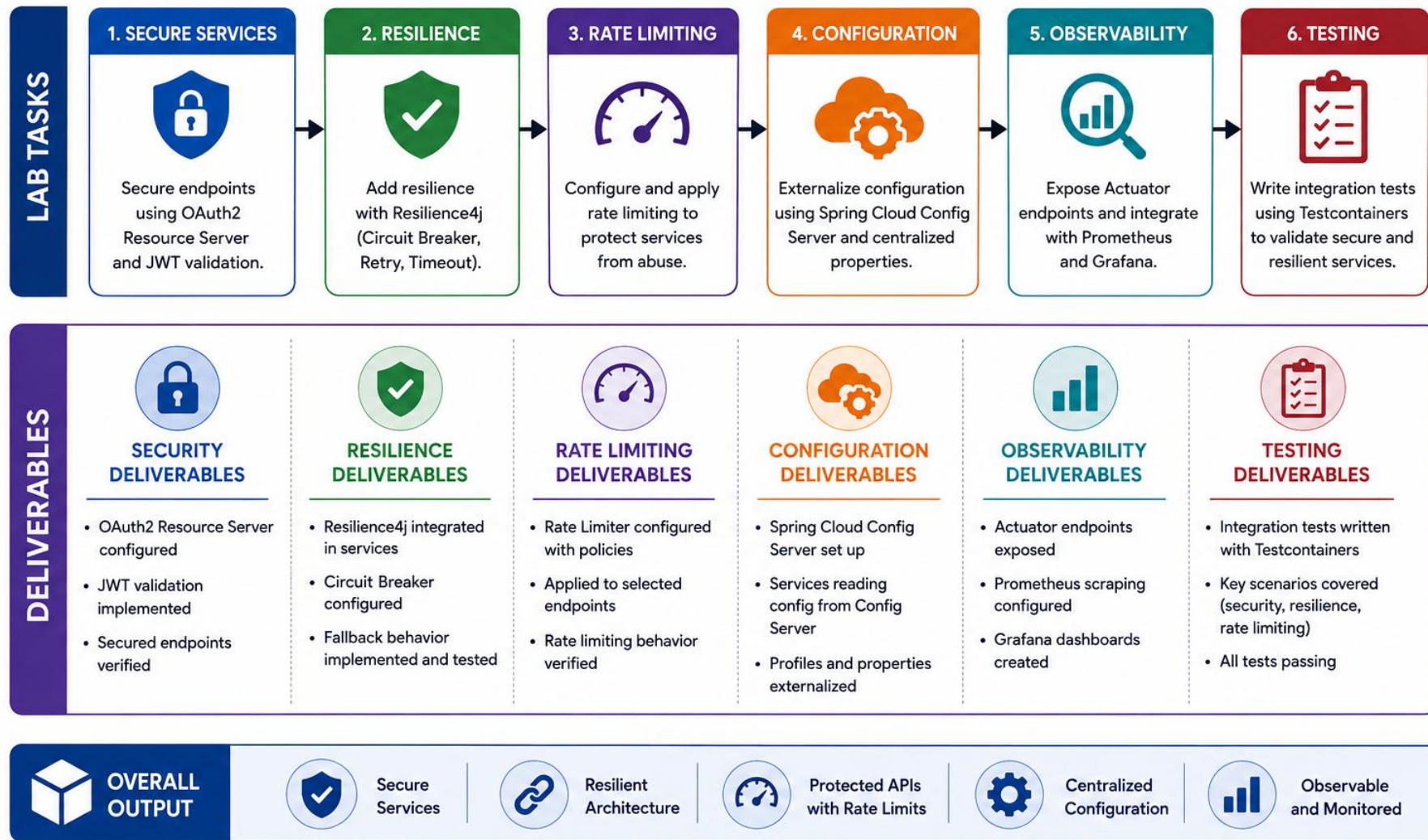


BEST PRACTICES

- Start with consumers -- write contract tests first
- Use a contract broker for sharing and discovery; verify contracts in CI for every provider change

KEY TAKEAWAY A Pact Broker becomes the shared source of truth, letting the consumer and provider teams release independently without breaking each other.

LAB TASKS AND DELIVERABLES



API COMPATIBILITY TESTING · END-TO-END TESTING · TEST DATA MANAGEMENT

TYPES OF COMPATIBILITY

- Backward compatible: new API works with existing consumers without changes
- Forward compatible: existing API can work with newer consumers
- Breaking change: existing consumers may fail with the new API

BEST PRACTICES

- Start with consumer-driven contracts; run provider verification in CI on every change
- Use semantic versioning for APIs; automate compatibility checks as a quality gate

WHAT E2E TESTING VALIDATES

- Complete workflows and integrations across service boundaries
- Data integrity, configuration, security, and environment issues
- Confidence before production releases

BEST PRACTICES

- Focus on critical business flows, not every path; keep tests independent and idempotent
- Tag tests (e.g., @E2E) and run less frequently than unit tests

TEST DATA LIFE CYCLE

- Identify -> Create/Acquire -> Store/Manage -> Provision -> Use & Refresh -> Retire/Archive

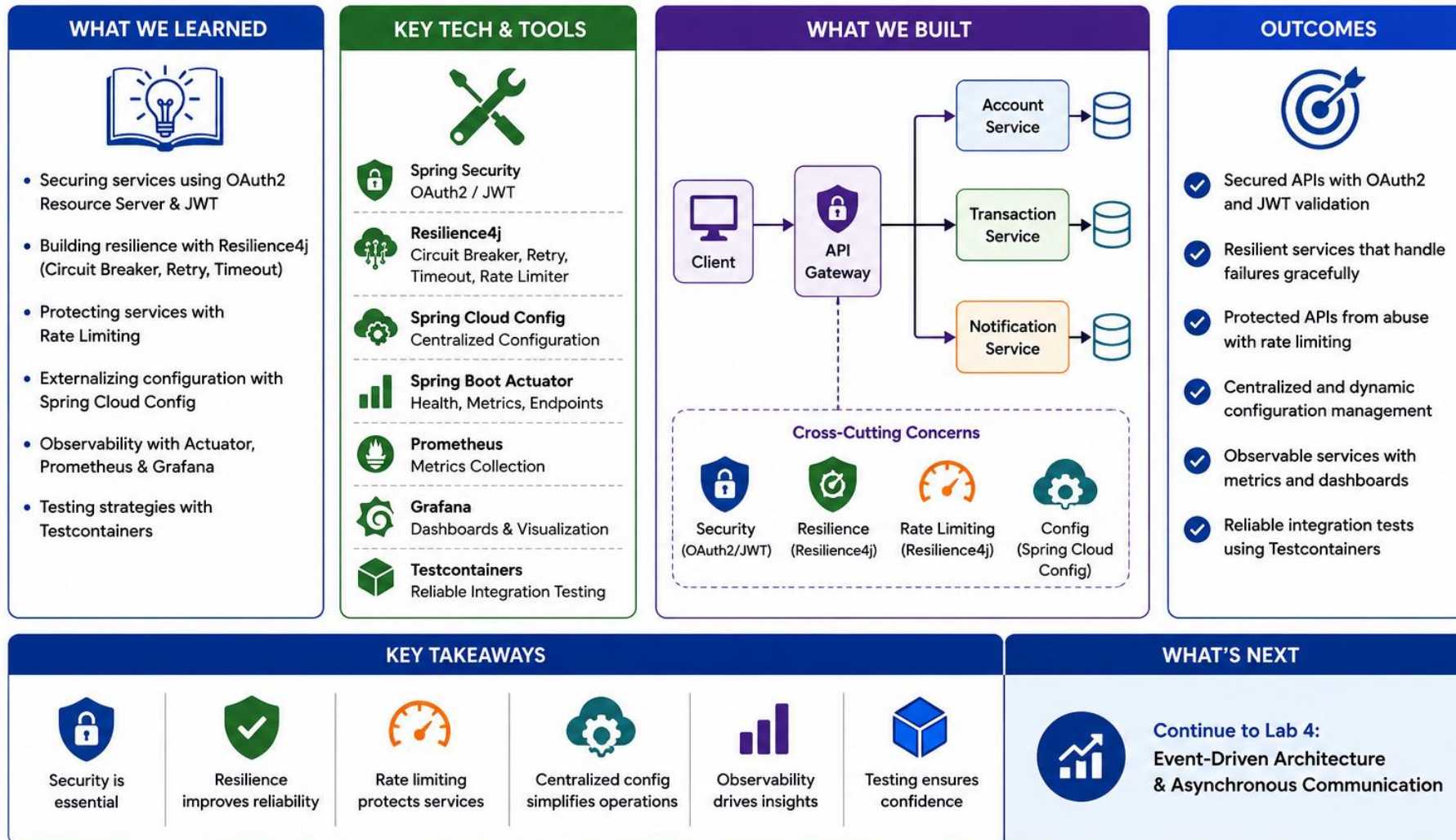
BEST PRACTICES

- Use Testcontainers to spin up clean, isolated databases
- Mask/anonymize sensitive data (PII) before use; clean up data after tests to avoid contamination

KEY TAKEAWAY Comparing a new API version against its baseline catches whether an added field, removed field, or changed type will break existing consumers. • Order creation flowing through the API gateway, order, inventory, payment, and notification services proves the whole system works together. • Production data must be masked before use; synthetic and reference data avoid that risk entirely while still exercising real scenarios.

DAY 3 SUMMARY

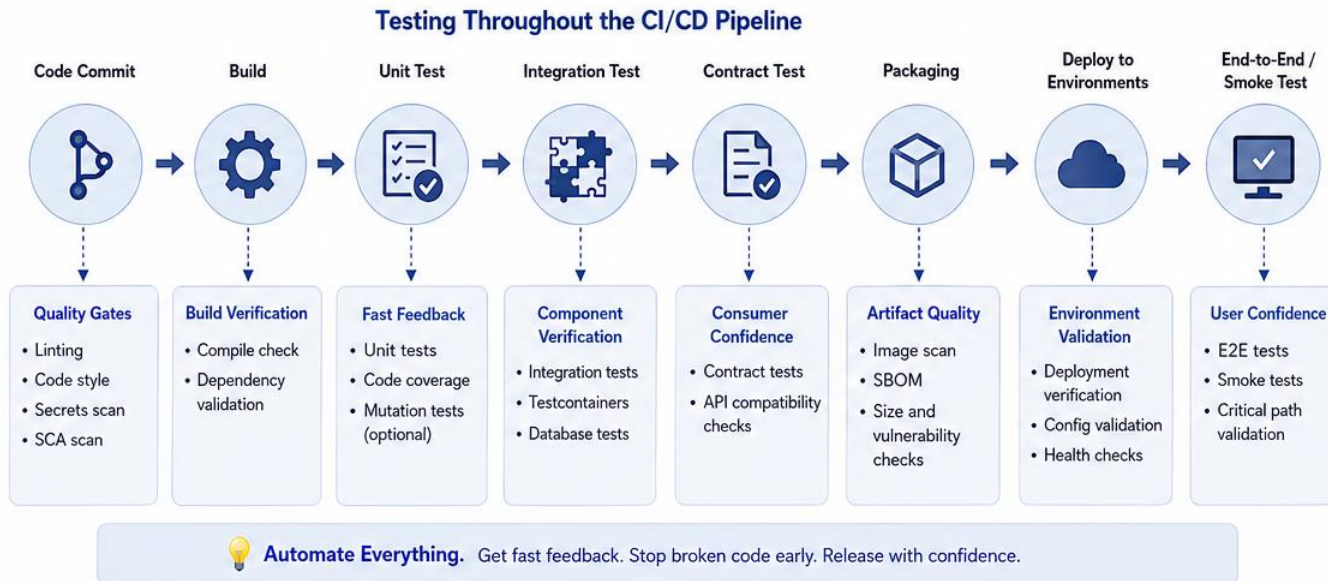
Secure, Resilient & Production-Ready Services





Continuous Testing in CI/CD

Continuous Testing shifts testing left and runs automated tests at every stage of the pipeline to deliver **faster feedback, higher quality, and confident releases.**



Benefits of Continuous Testing



Early Defect Detection

Find issues early in the pipeline when they are easier and cheaper to fix.



Faster Delivery

Automated tests enable faster, more frequent, and safer deployments.



Higher Quality

Continuous validation ensures better code, fewer defects, and happier customers.



Greater Confidence

Teams gain confidence to release changes quickly and reliably.



Lower Cost of Change

Fixing defects early reduces rework and production incidents.

Key Principles



Shift Left

Test as early as possible to catch defects when they are cheapest to fix.



Fast Feedback

Provide quick results to developers to enable rapid iteration.



Automate End-to-End

From unit tests to E2E tests—reduce manual steps and human error.



Quality Gates

Define and enforce quality thresholds at every stage of the pipeline.



Everyone Owns Quality

Quality is a team responsibility, not just the test team's job.



“Every commit is a potential release. Continuous Testing makes it a safe one.” — High-performing DevOps Teams



CODE COVERAGE AND TEST QUALITY · AVOIDING BRITTLE TESTS · PRODUCTION-READINESS TESTING CHECKLIST



GOOD COVERAGE TARGETS (GUIDELINES)

- Unit tests: 80-90% on core business logic and critical paths
- Integration tests: 70-80%; overall project: 70%+; critical modules: 90%+



COVERAGE VS. QUALITY

- High coverage, low quality: fragile, over-mocked tests give a false sense of security
- Good quality, meaningful coverage: resilient, focused tests that validate real behavior



WHY TESTS BECOME BRITTLE

- Overly coupled to implementation: depending on private methods, internal state, or call order
- Excessive mocking creates false confidence and breaks on refactoring
- Time and environment dependencies make tests unstable



STRATEGIES FOR RESILIENT TESTS

- Test behavior, not implementation; favor real components over mocks
- Control external factors with fixed clocks, Testcontainers, and in-memory DBs



CHECKLIST CATEGORIES (1-5)

1. Functional testing
2. Data & persistence testing
3. Integration testing
4. Resilience & reliability testing
5. Performance testing



CHECKLIST CATEGORIES (6-10)

6. Security testing
7. Observability testing
8. Configuration & environment
9. Quality & test coverage
10. Deployment readiness

KEY TAKEAWAY 80-90% unit, 70-80% integration, and 90%+ on critical modules are sensible coverage targets -- but meaningful coverage matters more than 100%. • Verifying internal method calls, over-mocking, and time-dependent assertions are the three most common ways tests become brittle. • Functional, data, integration, resilience, performance, security, observability, configuration, quality, and deployment readiness -- all ten, signed off.



KNOWLEDGE CHECK

1

What is the primary purpose of OAuth2 Resource Server?



- ☐ A To authenticate users locally
- ☐ B To validate JWTs and protect APIs
- ☐ C To issue access tokens
- ☐ D To encrypt request payloads

2

Which Resilience4j component prevents cascading failures?



- ☐ A Retry
- ☐ B Circuit Breaker
- ☐ C Rate Limiter
- ☐ D Timeout

3

What is the role of a Rate Limiter?



- ☐ A Encrypt HTTP traffic
- ☐ B Limit the number of requests over a period of time
- ☐ C Validate user roles
- ☐ D Cache responses

4

Which Spring Cloud Config feature allows centralized management of application properties?



- ☐ A Config Client
- ☐ B Config Server
- ☐ C Config Actuator
- ☐ D Config Repository

5

Which Actuator endpoint exposes application metrics by default?



- ☐ A /health
- ☐ B /info
- ☐ C /metrics
- ☐ D /loggers

6

Which tool is used to visualize metrics collected from services?



- ☐ A Kibana
- ☐ B Prometheus
- ☐ C Zipkin
- ☐ D Keycloak

7

What is Testcontainers primarily used for?



- ☐ A Container orchestration in production
- ☐ B Spinning up real dependencies in tests
- ☐ C Building Docker images
- ☐ D Monitoring container logs

8

Which HTTP status code indicates that the request was successful?



- ☐ A 400
- ☐ B 401
- ☐ C 200
- ☐ D 500

KEY CONCEPTS COVERED



Security
(OAuth2, JWT)



Resilience
(Resilience4j)



Rate Limiting
& Protection



Centralized
Configuration



Observability
(Actuator, Metrics)



Monitoring
(Prometheus, Grafana)

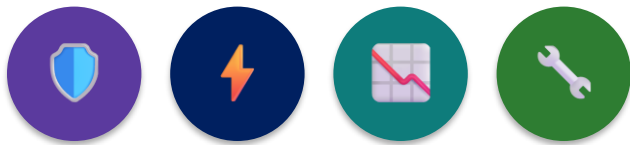


Testing
(Testcontainers)

LAB 3

Lab 3 — Secure and Resilient Services

Secured, resilient, tested Account and Transaction APIs with safe fallbacks.



LAB 3 — SECURE AND RESILIENT SERVICES — OVERVIEW

Day 3 lab: what you build and how it grows the capstone.



WHAT YOU WILL FINISH WITH

- Both APIs reject callers without a Bearer token (401)
- Wrong scope returns 403 (not a silent 200)
- Transaction Service forwards the caller's JWT when it GETs an account
- If Account Service is down, POST /api/v1/transactions returns 503 ACCOUNT_SERVICE_UNAVAILABLE — no Kafka event, no fake ACTIVE account
- After enough failures the circuit opens; later calls fail fast
- Unit tests for security and the circuit breaker
- Logs still use synthetic ids only — no Authorization header, no token payload

KEY TAKEAWAY Lab 3 builds Secure and Resilient Services -- the working code behind today's concepts, and the next layer of the banking capstone.

LAB 3 — SECURE AND RESILIENT SERVICES — BEFORE YOU START

Check these before the lab clock starts.

 KNOWLEDGE YOU NEED (FROM DAY 3)

Day 3 idea	How it appears in this lab
Resource server	Each API validates a JWT. It does not log users in with a form.
Scopes	accounts.read / accounts.write and transactions.read / transactions.write.
Workload vs user	The human caller holds the JWT. Transaction Service relays that token to Account Service.
Timeout	RestClient already uses a 2s connect / 3s read timeout. That is the timeout.
Circuit breaker	Resilience4j wraps AccountClient.requireActiveAccount. Repeated failures open the circuit.
Safe fallback	Fallback throws 503. It must not return a synthetic ACTIVE account.
Log redaction	Log accountId and status. Never log Bearer tokens or JWT claims.

KEY TAKEAWAY Settle prerequisites and environment first -- most lab time is lost to setup, not to code.

LAB 3 — SECURE AND RESILIENT SERVICES — LAB STEPS

6 steps, worked in order.

LAB STEPS

- Step 1 — Start both services and prove they are still open
- Step 2 — Protect Account Service with JWT
- Step 3 — Protect Transaction Service the same way

LAB STEPS (CONTINUED)

- Step 4 — Relay the Bearer token to Account Service
- Step 5 — Circuit breaker and safe fallback
- Step 6 — Tests and log review

KEY TAKEAWAY Work the steps in order -- each one builds on the last.

LAB 3 — SECURE AND RESILIENT SERVICES — SUCCESS CRITERIA

You are done when every box below is ticked.

SUCCESS CRITERIA

- No token → 401 on both business APIs
- Teller cannot POST transactions (403)
- Ops can POST a transaction for an ACTIVE account
- Token is forwarded; Account GET is authorized

SUCCESS CRITERIA (CONTINUED)

- Account down → 503, no auto-approve
- Circuit opens after repeated failures
- mvn test passes on both services
- Logs stay synthetic and secret-free

KEY TAKEAWAY Tick every criterion before moving on -- the next lab builds directly on this one.

"Secured, resilient, tested Account and Transaction APIs with safe fallbacks."

DAY 3 COMPLETE

Resilience, Security, and Testing

Day 3 complete — Secured, resilient, tested Account and Transaction APIs with safe fallbacks.

